

Estruturas fundamentais



SENAI

SENAI – DEPARTAMENTO REGIONAL DE SANTA CATARINA

MANTENEDORA

Fabrizio Machado Pereira
Diretor Regional do SENAI/SC e Diretor de
Educação e Tecnologia da FIESC

Adriana Paula Cassol
Gerente Executiva de Educação

Ana Luisa Mulbert
Cleunisse Aparecida Rauen de Luca Canto
Gestão da Educação Superior

CENTRO DE EDUCAÇÃO DIGITAL

Fabiano Bachmann
Gerente do Centro de Educação Digital - CDI

Gisele Umbelino
Coordenadora de Desenvolvimento de
Recursos Digitais - CDI

Hellen Cristine Geremia
Líder de Projeto

Gustavo Lucas Alves
Analista de Educação Digital

João Carlos Testi Ferreira
Autoria

Marlow Rodrigo Becker Dickel
Tiago Siqueira Asp
Revisor Técnico

Stephanie Johansen Longo Basso
Design Educacional

Heliziane Babosa
Design Gráfico

MANTIDA

Barbara Yadira Mellado Perez
Pro-reitora de Ensino, Pesquisa e Extensão
do Centro Universitário

Fabricio Roulin Bittencout
Diretor Faculdade Senai

Anderson da Rosa Eloy
Ilustrações e Tratamento de Imagens

Felipe Moisés da Silva Hintz
Programação Web

André Gonçalves de Freitas
Isac Diniz
Pedro Borrionuevo Nascimento
Produção Audiovisual

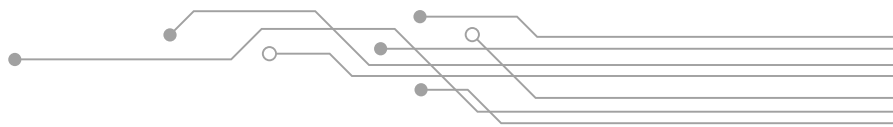
Michele Antunes Corrêa
Stephanie Johansen Longo Basso
Projeto Educacional

Heliziane Barbosa
Janaina da Silveira Vieira
Juliana Tonietto
Projeto Webgráfico

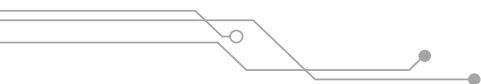
MSL Traduções
Revisão ortográfica, gramatical e
normalização

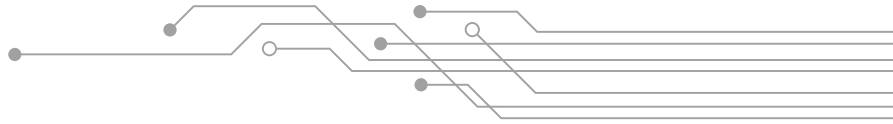
IStock Photo
Banco de Imagens

SUMÁRIO

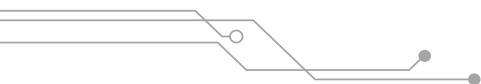


Estruturas fundamentais.....	1
Para Iniciar.....	5
Desafio.....	7
Desafio: Tipo Abstrato de Dados	9
Agenda Desafio: Tipo abstrato de dados	13
Estudo e Prática I: Linguagens e módulos	15
E-book I.....	15
Linguagens de programação	17
E-book II	39
Representação dos dados	41
Videoflix.....	67
O compilador C.....	67
Comparando linguagem de alto nível e baixo nível	67
Conversando sobre tipos em C	67
Manipulando memória com C	68
Tipos homogêneos e strings	68
Criando bibliotecas	68
Infocast.....	69
Tipos simples e ponteiros	71
Tipos heterogêneos	75
Comandos de manipulação de <i>strings</i> e o <i>printf</i>	79
Quero saber +	83
Resumindo	87
Estudo e Prática II: Estruturas básicas	89
E-book I.....	89
Estruturas estáticas básicas	91
E-book II	109
Estruturas básicas usando listas encadeadas	111
Videoflix.....	129
Lista baseada em vetores	129
Manipulando estruturas com nodos simples	129
Manipulando estruturas de nodo duplo	129





Infocast	131
Pilha sob vetor	133
Pilha sob lista encadeada	137
Quero saber +	143
Resumindo	145
 Para Concluir	 147





Olá, estudante!

A partir de agora, confira as estruturas básicas usadas em programação. Iniciaremos conhecendo um pouco sobre linguagens de programação e como elas se diferenciam.

Em seguida, para criar estas estruturas básicas, conheceremos a Linguagem C, suas características e formas de construção de programas.

Uma característica comum em linguagens é o poder de trabalhar com agrupamento de dados. Estudaremos essas formas de agrupamento, tais como tipos homogêneos e tipos heterogêneos. Identificaremos como usá-los e teremos a oportunidade de fazer uma aplicação prática envolvendo este aspecto para que esses conceitos sejam bem fixados.

Serão apresentados os fundamentos de estruturas, como pilhas, filas e listas, além de suas implementações estáticas e dinâmicas. Você também conhecerá aspectos de uso de funções, como passagem de parâmetros por valor e referência, e como isso acontece no uso das estruturas de dados.

Você também terá a oportunidade de se apropriar do conceito de Tipos Abstratos de Dados e como implementá-los criando bibliotecas funcionais e reutilizáveis. Para isso, usaremos a Linguagem C, porque ela trabalha em um nível de abstração um pouco mais baixo que outras linguagens e permite a manipulação de endereços por meio de ponteiros, o que ajudará na compreensão do funcionamento das estruturas em geral.

Se você quiser construir códigos eficientes, atendendo aos requisitos dos clientes, precisa conhecer mais a fundo o funcionamento das estruturas e utilizá-las de forma a alcançar seus objetivos. Assim, você conseguirá avaliar o desempenho de um algoritmo, o que irá auxiliar nos processos de escolha e de produção de códigos melhores.

Vamos iniciar essa jornada?



Confira, a seguir, uma situação prática relacionada aos conteúdos que você estudará a partir de agora. **Essa é uma importante etapa de seu processo avaliativo.** Então, dedique-se ao máximo e busque, em sua trajetória de estudos, fundamentar os resultados esperados no desafio que está proposto a seguir. Aproveite, pois essa é uma grande oportunidade de praticar novos conhecimentos!

Desafio - Tipo Abstrato de Dados

Vamos trabalhar em um exemplo real de criação de biblioteca para um objetivo específico, situação comum que você encontrará na construção de aplicações profissionais.

Essa atividade demandará que você avalie o problema e identifique a melhor solução de acordo com os requisitos nele estipulados.

Lembre-se: esteja atento a todos os detalhes indicados no Desafio e na Agenda, pois esta é uma parte importante de seu processo avaliativo.



Desafio: Tipo Abstrato de Dados



Uma organização está construindo uma aplicação que recebe milhares de requisições para atendimento de saúde. Essas requisições possuem as seguintes informações:

- Nome do Paciente – uma *string* de 40 caracteres
- Código de inscrição que o identifica no sistema de saúde – um número de tipo inteiro
- Código do procedimento solicitado – uma *string* de 10 caracteres

Essas requisições precisam ser organizadas por ordem de chegada e são consumidas pela aplicação na medida que se consegue alocar o paciente na instituição que irá atendê-lo. Assim, é necessário a criação de um **TAD** (Tipo abstrato de dados) que permita a inserção desta requisição, que será realizada quando ela chegar, e permita a remoção dela quando a aplicação conseguir alocá-la em uma instituição.

Outra característica necessária do TAD é fornecer a quantidade de requisições de espera, pois, conforme o tamanho, as instituições parceiras da organização também podem ser usadas para acelerar o processo de atendimento dos pacientes.

A construção precisa ser em linguagem C e, ao final de sua elaboração, devem constar dois arquivos:

- O cabeçalho do TAD denominado `estrutura.h`
- A implementação do TAD com o código-fonte, denominado `estrutura.c`, e o compilado, `estrutura.o`.

A estrutura da requisição é fornecida pelos arquivos:

- `requisicao.h`
- `requisicao.o`

Para seus testes de validação de sua implementação, está sendo fornecido um exemplo de código com uma função `main`, que pode ser usada para verificar seu funcionamento: - **`teste.c`**

Este arquivo pode ser baixado assim como o desafio e a agenda. Este será o teste realizado na avaliação de seu código.

Mas, atenção! **Funcionar é um elemento básico para o desafio, mas não é suficiente.** A avaliação do desafio irá considerar os seguintes aspectos:

- O código funciona, apresentando os dados corretos no teste original proposto [`teste.c`].
- O código é otimizado, consumindo o mínimo de memória (não há variáveis desnecessárias, não estão sendo armazenados dados temporários desnecessariamente?).
- O código é otimizado no tempo de resposta de inserção e remoção (O código está com a menor complexidade possível para o caso?).
- O código atende a todos os requisitos demandados.

TESTE.C

```
#include "requisicao.h"
#include "estrutura.h"
#include <stdlib.h>
#include <stdio.h>
#include <strings.h>

void imprime(Requisicao *requisicao) {
    printf("-----\n");
    printf("Endereco: %p\nNome      : %40s\nInscricao : %40d\nProcedimento: %40s\n\n", requisicao, get_nome(requisicao), get_inscricao(requisicao), get_procedimento(requisicao));
}

int main() {
    char nomes[][40] = {"Joao", "Jose", "Maria", "Silvia", "Sonia", "Pedro", "Lucas", "Bernardete", "Lucia", "Olivia"};
    char procedimentos[][20] = {"123.1", "345.6", "678.9", "098.1", "758.5", "23.4", "5.98", "4.32", "3.3", "1.4"};
    Estrutura *estrutura = create();
    int i;
    Requisicao *requisicao;
    for(i = 0; i < 10; i++) {
        requisicao = cria_requisicao(nomes[i], i, procedimentos[i]);
        inserir(estrutura, requisicao);
        printf("Insercao de %40s, quantidade na estrutura: %04d\n", get_nome(requisicao), get_size(estrutura));
    }
    printf("Fim da insercao.\nInicio da remocao\n");
    for(i = 0; i < 10; i++) {
        Requisicao *r = remover(estrutura);
        printf("Removido  %40s, quantidade na estrutura: %04d\n", get_nome(r), get_size(estrutura));
        imprime(r);
        libera(r);
    }
    return 0;
}
```


AGENDA | Tipo abstrato de dados

Resultado esperado

Arquivo zip contendo os arquivos produzidos em linguagem C, cumprindo os aspectos do desafio e critérios de avaliação.

Desenvolvimento

Individual. Confirmar com professor tutor.

Critérios de avaliação

- Apresentar o cabeçalho do TAD denominado estrutura.h.
- Inserir a implementação do TAD com o código-fonte, denominado estrutura.c, e o compilado, estrutura.o.
- Mostrar requisicao.h e requisicao.o.
- Expor que o código funciona, apresentando os dados corretos no teste original proposto [teste.c].
- Desenvolver o código otimizado, consumindo o mínimo de memória (não há variáveis desnecessárias, não estão sendo armazenados dados temporários desnecessariamente?).
- Aplicar o código, otimizado no tempo de resposta de inserção e remoção (O código está com a menor complexidade possível para o caso?).
- Evidenciar que o código atende a todos os requisitos demandados.
- Entregar atividade conforme prazo estabelecido.

Forma de entrega

Arquivo em formato.zip com o agrupamento dos arquivos produzidos. O arquivo em .zip deverá conter seu nome. Exemplo: o aluno José Augusto Seabra cria o arquivo JoseAugustoSeabra.zip, a ser entregue em ferramenta do Ambiente Virtual de Aprendizagem (AVA).



A tecnologia da informação está por toda parte. Essa é uma área que só cresce em demanda e oportunidades. Basicamente essa área envolve um hardware, um programa de computador e uma estrutura de dados de suporte. O que tratamos aqui é um destes pilares: a estrutura dos dados.



E-book

Aqui você irá encontrar dois e-books. No primeiro, será abordada a linguagens de programação - conceitos, características, vantagens e desvantagens; linguagem C - uma introdução; compiladores: funcionamento, formas de execução, produtos intermediários da compilação. No e-book seguinte, você estudará os tipos de dados, modos de alocação de memória, manipulação de memória, tipos de dados homogêneos e heterogêneos e tipos abstratos de dados.

Vários elementos são importantes no estudo das estruturas de dados. Agora confira alguns conceitos de linguagens de programação, suas características de uso e linguagem C, usada para a compreensão de estrutura de dados. Conheça também os compiladores, seu funcionamento básico e as formas de execução. Para trabalhar com estruturas de dados, são necessários meios de suporte para estas estruturas. Para isso, é preciso conhecer o uso de ponteiros, tipos homogêneos e tipos heterogêneos. Estes tipos são os mais usados para sustentar as estruturas de dados. Finalmente precisa-se estabelecer os tipos abstratos de dados, as estruturas especiais que são criadas com objetivo específico, muito usadas na criação de bibliotecas.

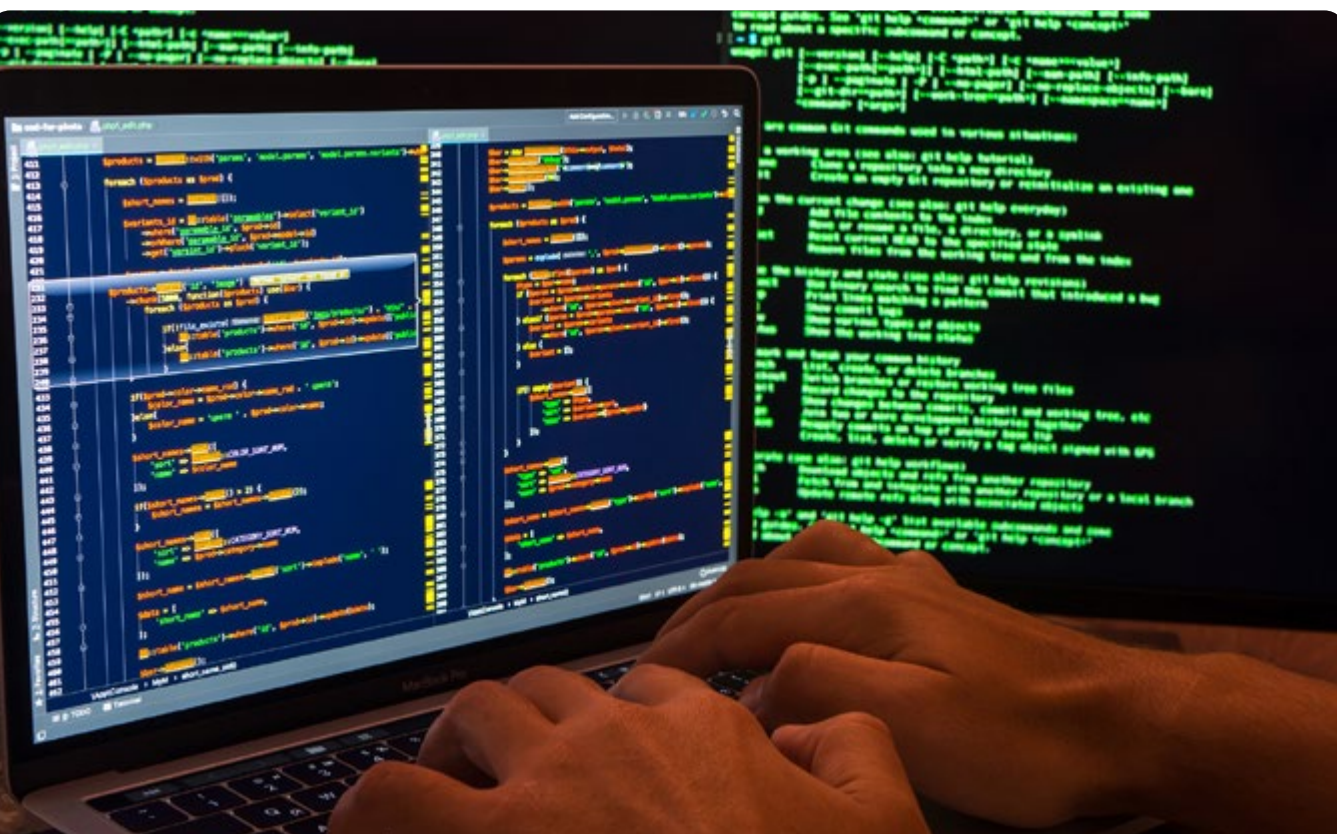


Linguagens de programação



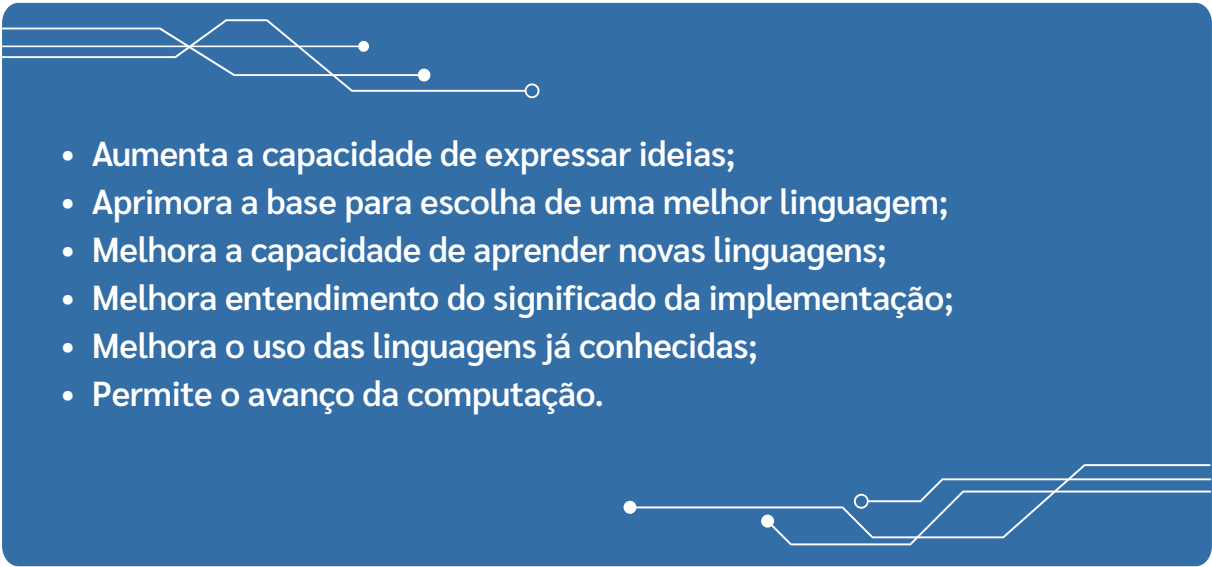
QUAIS LINGUAGENS DEVEMOS CONHECER?

Para saber programar, é preciso conhecer o que torna uma linguagem de programação diferente da outra. Afinal, por que há tantas linguagens de programação? É preciso conhecer mais de uma linguagem?



Para conhecer a estrutura de dados usando a Linguagem C, você precisa aprender a trabalhar com seu compilador.

Conhecer linguagens de programação ajuda a produzir softwares melhores. Conforme Sebesta (2012), há várias razões para se estudar os conceitos sobre linguagens de programação. O autor cita os benefícios obtidos com este estudo:

- 
- Aumenta a capacidade de expressar ideias;
 - Aprimora a base para escolha de uma melhor linguagem;
 - Melhora a capacidade de aprender novas linguagens;
 - Melhora entendimento do significado da implementação;
 - Melhora o uso das linguagens já conhecidas;
 - Permite o avanço da computação.

Cada linguagem possui características quanto a tipos fornecidos, estruturas de dados disponíveis entre outras características que estão relacionadas a seus aspectos gerais. Saiba um pouco mais sobre as características das linguagens.

CARACTERIZANDO LINGUAGENS

É possível agrupar linguagens conforme o seu domínio ou uso. Para Sebesta (2012), existem os seguintes domínios: aplicações científicas, aplicações de negócios, inteligência artificial, programação de sistemas operacionais ou suporte e aplicações WEB. Além destes domínios, há também as aplicações de lazer, como os jogos.

As linguagens são construídas para resolver uma dificuldade de uso ou computacional qualquer. Sebesta (2012) cita três características que permitem avaliar uma linguagem: sua legibilidade, sua facilidade de escrita e sua confiabilidade. Para obter mais ou menos estas características, as linguagens precisam dar mais ênfase em algumas de suas formas de codificação. Além destes aspectos, Sebesta (2012) apresenta outros aspectos que influenciam o modelo da linguagem, tais como a arquitetura dos computadores e a metodologia de desenvolvimento.



Outra forma de caracterizar linguagens é pelo seu paradigma. Conforme Ferreira (2020), os paradigmas podem ser: imperativo, declarativo, estrutural, procedimental, funcional e orientado a objetos. Apesar de as linguagens serem caracterizadas pelo paradigma, na verdade o paradigma está relacionado à forma de pensar do programador, à maneira como ele desenvolve suas soluções. Ascencio e Campos (2012) observam que qualquer paradigma pode ser aplicado em qualquer linguagem, até mesmo em Assembly, o que irá depender de como o programador fez uso da linguagem.

Comparando tipos de linguagens

Para compreender melhor as características das linguagens, pode-se comparar linguagens com características de programação linear, programação estruturada e orientada a objetos. Isso permitirá identificar aspectos evolutivos das linguagens, modos de simplificação de reuso, controle de repetição de código, dentre outras características que tornam uma mais adequada do que outra, conforme o contexto de uso.

Programação linear

Linguagens que apresentam característica linear são aquelas que não possuem estruturas de controle (for, while, ...). A maneira de criar repetições no código é por meio de um comando do tipo “GO TO”. Seu código é uma sequência de comandos ou operações que só alteram a sua sequência de execução por uma chamada explícita para redirecionamento para uma determinada linha do código. Observe um exemplo de código com uma linguagem de alto nível antiga, o Basic.

```
5  REM COMANDO DE IMPRESSAO NA TELA
10 PRINT “Quanto e 7 - 7?”
15 REM LEITURA DA VARIABEL A
20 INPUT A
25 REM VERIFICA SE A EH 0, SE FOR VAI PARA 60 SENAO CONTINUA
30 IF A = 0 THEN GOTO 60
35 REM COMANDO DE IMPRESSAO NA TELA
40 PRINT “Errado... Tente outra vez!”
45 RETORNA PARA A LINHA 20
50 GOTO 20
55 REM COMANDO DE IMPRESSAO NA TELA
60 PRINT “Certo! Parabens!”
65 REM FIM DO PROGRAMA
70 END
```

Observe que, no Basic, cada linha é numerada, o que serve como um rótulo para orientar o GOTO. O comando REM é comentário e sua linha é ignorada na execução. Sua execução é sequencial e, conforme as condições na execução, o comando GOTO faz o redirecionamento para determinada linha.

À medida que o código cresce, fica bastante difícil compreendê-lo, acompanhar as idas e vindas nas linhas. Esse tipo de linguagem não favorece a manutenção, e o reaproveitamento de código para outros fins é praticamente impossível.



Curiosidade

Dado a característica de ir e vir no código, parecendo um emaranhado, este tipo de programação era chamado macarrão ou espaguete.

As linhas precisavam ser numeradas, pois a execução ocorria na sequência numérica das linhas. As linhas eram numeradas de 10 em 10, pois, se fosse necessário incluir um código entre o já existente, seria necessário renumerar todas as linhas posteriores. Como o salto já era feito de 10 em 10, era possível incluir linhas com os valores dentro dessa faixa, a exemplo do que foi feito com os comentários neste código.



Programação estruturada

Este tipo de programação se caracteriza pela organização do código em blocos. Além dos blocos caracterizando condicionais e laços, existem os procedimentos e as funções. Quando o bloco é uma função, podem ser passados parâmetros que são usados no interior do bloco e, ao final de sua execução, a função devolve um resultado. Quando o bloco é um procedimento, também podem ser passados parâmetros, mas o procedimento não devolve resultado.

Confira um exemplo de código em uma linguagem estruturada de alto nível, o Pascal.



```
program example;                                //Declaração de início do programa
    var                                          //Declaração das variáveis globais
    x:integer;
    y:integer;

function fat(n:integer): longint;
    //Declaração da função fat, com tipo de
    //retorno longint e parâmetro inteiro

var                                          //Declaração das variáveis locais da função
    i: integer;
    f: longint;

begin                                          //Início do bloco de código da função fat
    f:=1;
    for i:= 2 to n do
        f:= f * i;
        fat:= f;
end;                                          //Fim do bloco código da função fat

procedure troca(var v1, v2: integer);
    //Declaração do procedimento troca, com parâmetros inteiros
var                                          //Declaração das variáveis locais da função
    aux: integer;
```

```

begin                                //Início do bloco de código do procedimento troca
    aux := v1;
    v1 := v2;
    v2 := aux;
end;                                //Fim do bloco de código do procedimento troca
begin                                //Início do bloco de código do programa principal
    writeln('Digite varios numeros para saber o fatorial');
    writeln('Termine com -1');
    read(x);
    while x > 0 do

        begin                        //Início do bloco de código do while
            writeln('fatorial de ', x, ' = ', fat(x) );
            //Função fat sendo chamada no comando de impressão
            read(x);
        end;                        //Fim do bloco de código do while
        y := 50;
        troca(x, y);
        //Chamada do procedimento troca com seus argumentos
        writeln('O valor de x e ', x, ' e y e ', y );
    end.                            //Final do programa

```

Os comentários em Pascal podem ser feitos na própria linha, bastando iniciar por duas barras. Tudo o que vier depois das duas barras até o final da linha é ignorado. É possível observar diferenças significativas na forma de programar do Basic para o Pascal. O código não fica misturado, sendo que cada ação do programa tem seu próprio bloco de código. Isso facilita o posterior reuso. Mesmo normalmente sendo por copia e cola, neste caso, a função ou o procedimento serão copiados de forma completa, sem redirecionamento para alguma posição estranha no código.

No bloco da função *fat*, percebe-se que o nome da função é usado para receber o seu valor de retorno. Em Pascal, o nome da função é usado como variável de retorno, ou seja, ao terminar a execução, o valor que estiver em *fat* será o retorno da função.

Observe a diferença na chamada do *procedimento* e *da função*. Quando se chama a função, ela deve ser usada para atribuir seu resultado em uma variável ou servir de retorno para outra função. Neste caso, usa-se o seu resultado para impressão através da função *writeln*. Na chamada do *procedimento*, não há retorno. Logo, não há atribuições ou uso de resultado para algum outro fim.

Dica



Na declaração da *função* ou do *procedimento*, a declaração das variáveis fica entre parênteses após seu nome, e essas são chamadas parâmetros da função ou do procedimento. Quando de sua chamada para execução, os valores passados nos parâmetros são chamados argumentos.

Programação orientada a objetos

Este tipo de programação se caracteriza pela organização do código em elementos independentes (classes). Os aspectos de blocos estão presentes na programação orientada a objetos, mas extrapolam o conceito por criar unidades mais independentes, o que permite um reuso seguro destas unidades e sem a necessidade do copia e cola. Confira um exemplo na linguagem Java.

```
public class Conta {                                //Declaração da classe
    private String titular;                          //Declaração de atributos
    private int numero;
    protected double saldo;
```



```
public String getTitular() {  
    //Declaração de método getter  
    return this.titular;  
}  
public void setTitular(String titular) {  
    //Declaração do setter  
    this.titular = titular;  
}  
public int getNumero() {  
    return numero;  
}  
public void setNumero(int numero) {  
    this.numero = numero;  
}  
public double getSaldo() {  
    return saldo;  
}  
public void sacar(int valor) {  
    //Método de manipulação de atributo seguro  
    if((saldo - valor) < 0) throw new  
        SaldoInsuficienteException("Saldo insuficiente: " + saldo);  
    saldo -= valor;  
}  
public void depositar(int valor) {  
    saldo += valor;  
}  
}
```

Observando a classe, percebe-se que é um conjunto de código autocontido. Seus dados (*atributos*) possuem funções de manipulação (*métodos*) para sua manipulação, o que garante a segurança na operação dos atributos. Os atributos normalmente não são acessados diretamente. Para acessá-los, são usados seus métodos de acesso (*getter* e *setter*). Essa é apenas uma visão introdutória, sendo que mais de detalhe serão apresentados juntamente com os tipos abstratos de dados.

Dica



Em Java, os métodos são declarados sempre indicando o tipo de retorno, por exemplo, `public double getSaldo()` – *double* é o tipo de retorno. Mas, nem todos os métodos possuem retorno. E, para representar isso, há, por exemplo, `public void setNumero(int numero)` – ao usar `void` como tipo de retorno, diz-se que não há retorno. Assim, se o tipo de retorno é `void`, há um procedimento, e se há um tipo não `void` de retorno, esta é uma função.

FORMAS DE EXECUÇÃO

Existem basicamente três tipos de linguagem quanto à forma de execução: interpretadas, híbridas e compiladas. Aho *et al.* (2008) explicam a diferença entre as formas de execução. Acompanhe.

Processador de linguagem interpretada – conforme a figura, tem-se um interpretador que realiza o processamento da linguagem. O interpretador executa o programa-fonte sobre os dados fornecidos, gerando a saída. Um exemplo de linguagem que funciona assim é o Python.



Figura 1 - Um interpretador
Fonte: Adaptado de Aho et al. (2008)

Processador de linguagem híbrida – conforme a figura a seguir, o programa-fonte é inicialmente compilado para uma linguagem intermediária (bytecodes), para depois ser interpretado por uma máquina virtual. A máquina virtual processa o código intermediário sobre os dados fornecidos e gera a saída. Um exemplo de linguagem que funciona assim é o Java.

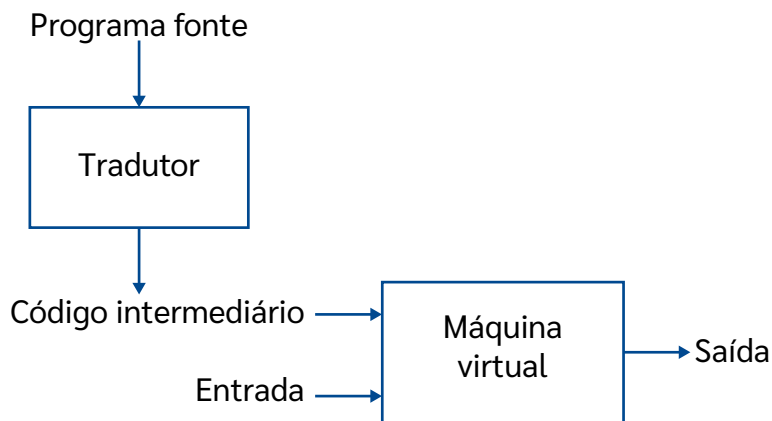
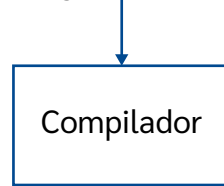


Figura 2 - Um compilador híbrido
Fonte: Adaptado de Aho et al. (2008)

Linguagem compilada – o código-fonte, neste caso, não pode ser submetido à execução antes de ser traduzido para a linguagem nativa da máquina. Primeiro passa-se o código-fonte por um compilador, conforme mostra a figura a seguir. A saída é um programa-objeto em linguagem de máquina. O processo de compilação envolve algumas etapas, que serão apresentadas quando se discutirá o compilador da linguagem C.

Programa-fonte



Programa-objeto

Figura 3 - Um compilador
Fonte: Adaptado de Aho *et al.* (2008)

Sua execução acontece pela chamada do programa-objeto, que recebe entradas e fornece saídas, mostrado na figura a seguir.. Não há, nesse caso, o uso de um interpretador.

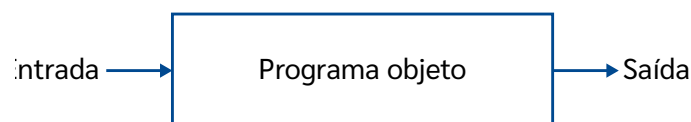


Figura 4 - Chamada do programa-objeto
Fonte: Adaptado de Aho *et al.* (2008)

Programas compilados normalmente são mais performáticos, mas a codificação do fonte varia conforme a máquina hospedeira. Quando se usa interpretadores, o código não muda, pois é feito para o interpretador ou máquina virtual, independentemente da máquina hospedeira.

LINGUAGEM C

Conforme Kernighan e Ritchie (1989), a linguagem C é de uso geral e tipada, possuindo como tipos fundamentais: o caractere, o inteiro e números de ponto flutuante. Além dos tipos fundamentais, possui outros como ponteiros, vetores, estruturas, entre outros.

Nestes estudos, a linguagem C será apresentada devido à sua característica facilitada de manipulação da memória. Você conhecerá essa linguagem com exemplos explicados. O primeiro exemplo apresenta a estrutura básica da linguagem. Acompanhe.

```

int main() {
    //Declaração da função main que devolve um inteiro
    //como retorno e início do bloco de código

    int a = 10;    //Declaração da variável a inicializada com 10
    int b = 20;    //Declaração da variável b inicializada com 20
    return a + b;

    //retorno da função com a soma das variáveis a e b
}                //Final do bloco da função

```

Quando se quer usar uma função já existente em algum módulo, seja da linguagem ou não, é preciso incluir no código-fonte o arquivo de cabeçalho (*Header*) daquele módulo. No arquivo de cabeçalho, estão declarados os tipos e as funções do módulo, para que o compilador consiga fazer a avaliação em busca de erros antes de traduzi-los para o programa-objeto. Para incluir o arquivo de cabeçalho, usa-se a diretiva de pré-processamento *#include*. Acompanhe um exemplo.

```

#include <stdio.h>
int main() {
    printf("%s\n", "ola mUndo.");
    return 0;
}

```

No local onde se usa a diretiva *#include* é onde será inserido o conteúdo do arquivo de cabeçalho, no caso do exemplo, é o arquivo `stdio.h` da biblioteca do C. Este módulo contém as funções de entrada e saída (input/output, ou simplesmente I/O). O arquivo está cercado de `<>` para indicar que o compilador deve procurar o arquivo no diretório padrão configurado pelo instalador do compilador. Se o arquivo estiver em local diferente, o nome do arquivo deve ser indicado entre aspas e não cercado de `<>`.

Outra diretiva de pré-processamento muito usada é a *#define*. Ela funciona como um meio de buscar e substituir. Confira um exemplo.

```
#include <stdio.h>
#define LENGTH 5
int main() {
    int i;
    for(i = 0; i < LENGTH; i++) {
        printf("Passo %d\n", i);
    }
    return 0;
}
```

No pré-processamento, onde houver `LENGTH`, será substituído por 5. Isso é especialmente útil quando há valores padronizados usados em várias partes do código. Quando estes valores mudam, basta alterar no *#define* e o restante do código considerará o novo valor – isso permite colocar uma palavra que dê significado para um número que ficaria fixo.

O compilador GCC

O compilador faz a tarefa de pegar um código-fonte e transformá-lo em um programa-objeto executável por meio de etapas. Aho *et al.* (2008) comenta sobre as quatro etapas do compilador, conforme apresentado na figura, a seguir.

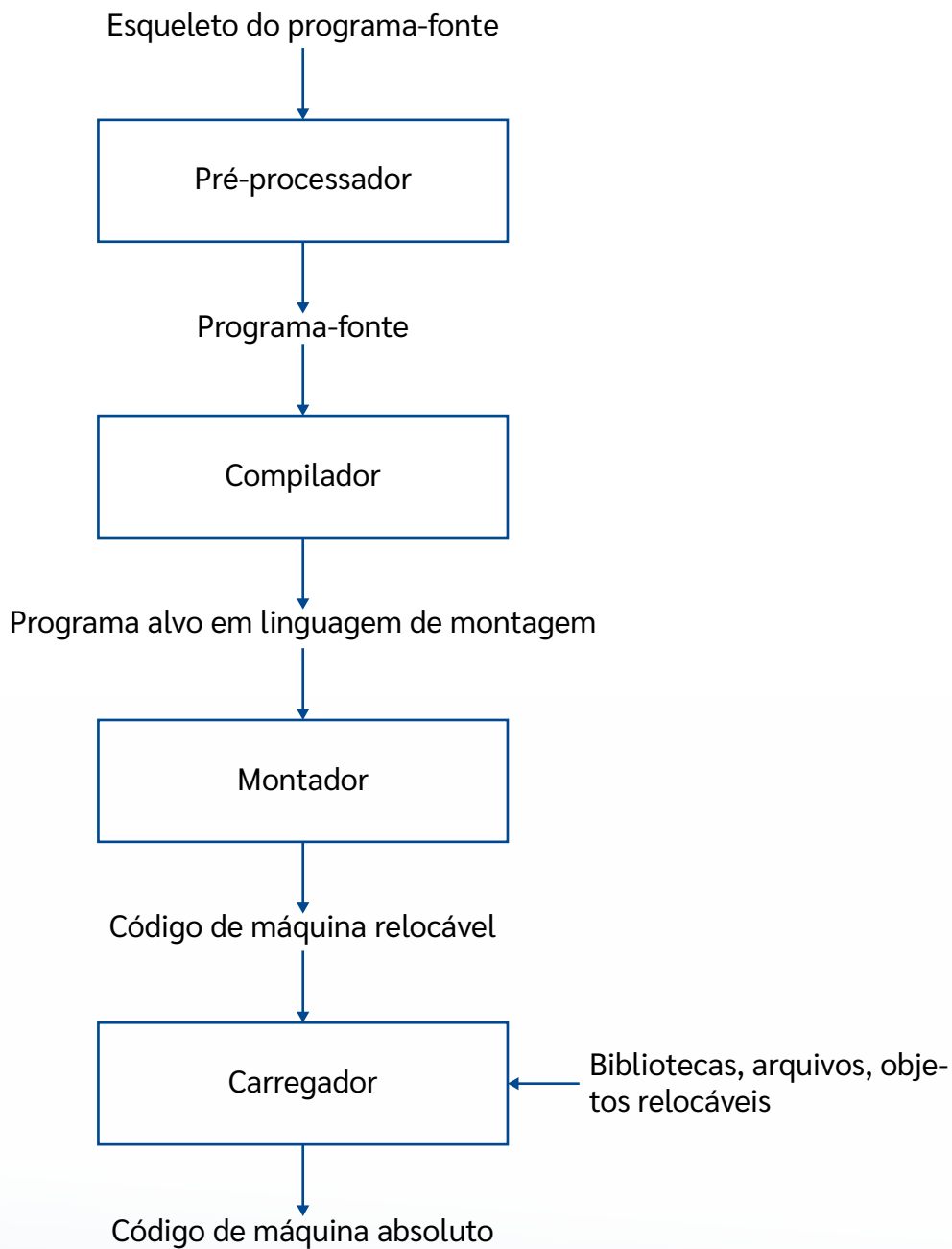


Figura 5 - Sistema de processamento de linguagem
Fonte: Aho *et al.* (2008)



Pré-processador – realiza o tratamento prévio do código-fonte, incluindo cabeçalhos, fazendo substituições com *define* ou mesmo realizando macros.

Compilador – recebe o código-fonte modificado pelo pré-processador e o traduz para uma linguagem simbólica intermediária (*Assembly*).

Montador – o montador (*Assembler*) pega a saída em linguagem simbólica e produz uma saída em código de máquina relocável. Este resultado não é executável, mas é usado para construir módulos que podem ser usados por outros programas.

Editor de ligação/Carregador – o código de máquina precisa ser ligado às bibliotecas de código relocável que estão sendo usadas pelo programa. O editor de ligação (*linker*) faz isso, resolvendo os endereços corretos para a aplicação, e o carregador (*loader*) reúne tudo e o torna executável.



Para compilar os códigos em linguagem C, usa-se o GCC (GNU Compiler Collection). Há vários compiladores para linguagem C. A opção pelo GCC se deve ao fato de ele ser um software livre. O GCC normalmente vem instalado no LINUX, mas não no Windows.

Dica



Para usuários Windows, você pode aprender como instalar o GCC em links que estão disponíveis no item Quero Saber Mais.

A edição do código-fonte pode ser feita por qualquer editor de texto puro (bloco de notas do Windows, por exemplo).

Dica



Um excelente editor de código é o `vscode`, que você pode acessar no item [Quero Saber Mais](#).

Com o GCC instalado e operacional, para executar o compilador GCC, há várias opções. Serão apresentadas as mais usadas, todas realizáveis através do prompt de comando do sistema operacional, considerando que o arquivo que contém o código a ser compilado se chama `fonte.c`.

```
$ gcc -E fonte.c
```

O `$` representa o prompt da linha de comando. A opção `-E` realiza apenas o pré-processamento do código do arquivo passado. A saída será na tela.

```
$ gcc -S fonte.c
```

A opção `-S` converte o código do arquivo `fonte.c` para linguagem simbólica – Assembly. O resultado é um arquivo de mesmo nome, mas com extensão `.s`, neste caso, seria `fonte.s`.

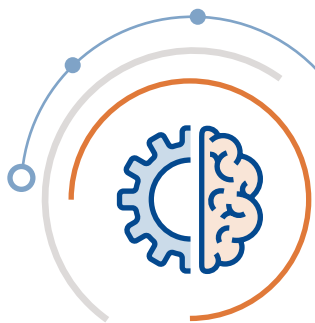
```
$ gcc fonte.c
```

Executando sem opções, o compilador executa todas as etapas gerando na saída o arquivo executável. O arquivo de saída terá o nome a.out – se o ambiente for Linux/Unix, ou a.exe – se o ambiente for Windows.

```
$ gcc fonte.c -o fonte
```

Para que o arquivo compilado tenha um nome específico, pode-se usar a opção -o. Com isso, o arquivo de saída será aquele fornecido após a opção -o. No Linux, o nome do arquivo será exatamente o fornecido; se for Windows, o nome terá a extensão .exe incluída.

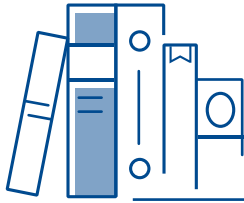
Na galeria de vídeos, assistindo o vídeo **O compilador C**, você poderá entender o funcionamento do compilador com alguns exemplos.



Mão na massa

Foram apresentados vários códigos e exemplos em C. Crie arquivos de código-fonte com cada um deles e realize a compilação com cada uma das opções para verificar o seu funcionamento.

Conhecer as características das linguagens, sua forma de execução, vantagens e desvantagens de cada uma dessas características, permite que você escolha a melhor opção de linguagem no desenvolvimento de um projeto. A linguagem C é uma das mais usadas no mundo, e você acompanhou seus elementos básicos e o compilador GCC, com suas principais formas de execução, que serão necessárias durante todo o processo de aprendizagem de estruturas de dados com a linguagem C.



REFERÊNCIAS

SEBESTA, R. W. **Concepts of programming languages**. 10. ed. Upper Saddle River: Pearson, 2012.

FERREIRA, R. D. **Linguagem de programação**. Curitiba: Contentus, 2020.

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamento da programação de computadores**: algoritmos, pascal, c/c++ (padrão ansi) e java. 3. ed. São Paulo: Pearson do Brasil Education, 2012.

AHO, A. V. *et al.* **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

KERNIGHAN, B. W.; RITCHIE, D. M. C, **a linguagem de programação**: padrão ansi. 2. ed. Rio de Janeiro: Campus, 1989.

SENAI



E-book

Para o computador poder trabalhar com as coisas do mundo real, é necessário haver algum tipo de representação destas coisas. Para construir bons programas, é preciso compreender como essas coisas são representadas para poder tirar o melhor proveito delas. A seguir, confira como entender e apropriar-se da representação dos dados, para que você tenha entendimento sobre tipos de dados, modos de alocação de memória, manipulação de memória, tipos de dados homogêneos e heterogêneos e tipos abstratos de dados.

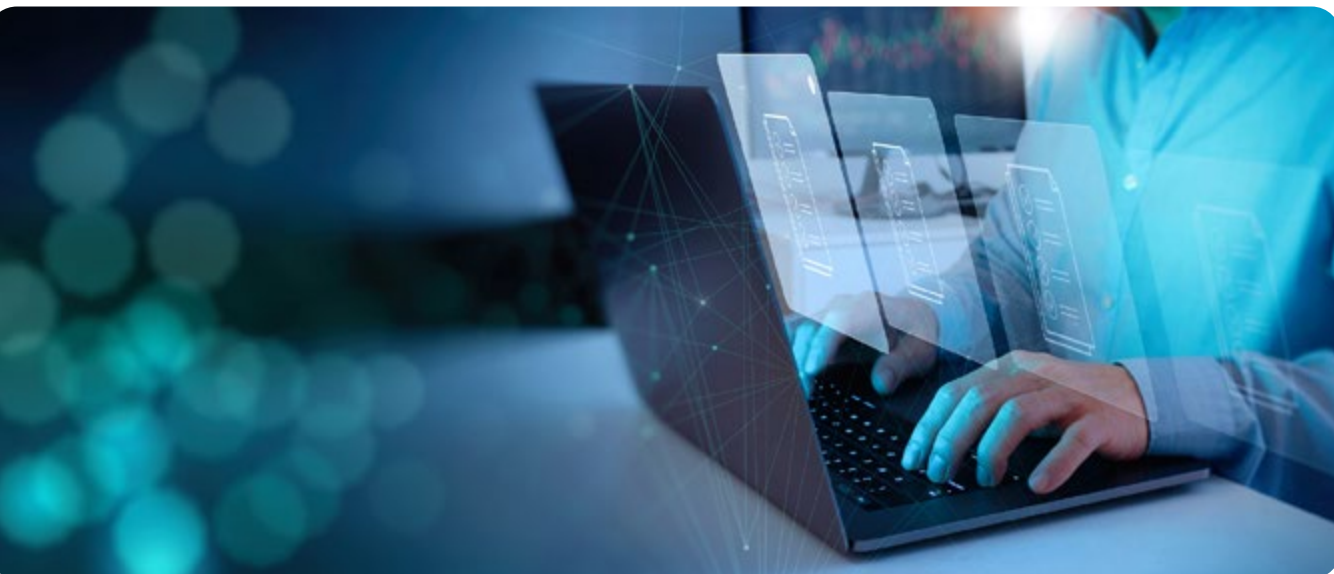


Representação dos dados



INTRODUÇÃO

Um computador é uma máquina que pode fazer praticamente qualquer coisa que se deseja, mas, para isso, é necessário criar representações (abstrações) das entidades do mundo real para uma forma computacional viável. Conforme Moraes (2001), há limitações sobre o que o computador pode representar. E, por isso, é preciso criar essas abstrações, para simplificar estes elementos em suas partes essenciais.



London (2000) comenta que é preciso representar dados e estruturas de todas as formas e tamanhos. E, para isso, é usado algum tipo de abstração dessas estruturas. O uso de estruturas de dados, conforme London (2000), possui três motivos: eficiência, abstração e reutilização.

No aspecto de eficiência, London (2000) destaca que a estrutura de dados favorece a eficiência do algoritmo, seja pela forma de organizar os dados ou pela facilidade de sua manipulação. Com relação à abstração, London (2000) cita que a estrutura de dados é uma representação compreensível de como se vê os dados, facilitando o entendimento e a solução de um problema. Já quanto à reutilização, London (2000) destaca que estruturas de dados normalmente são modulares e isso permite que sejam usadas nas mais diversas soluções que empregam o mesmo tipo de abstração.

TIPOS DE DADOS

No uso de programas computacionais, há diversos tipos de dados, por exemplo, dados numéricos, *strings* (cadeias de caracteres), e assim por diante. Cada linguagem possui algum tipo de particularidade com relação aos tipos de dados. Na linguagem C, Kernighan e Ritchie (1989) citam os tipos básicos disponíveis na linguagem:

- *char*: com o tamanho de um byte, pode representar um caractere;
- *int*: um inteiro, normalmente do tamanho de um inteiro natural na máquina hospedeira;
- *float*: número de ponto flutuante de precisão simples;
- *double*: número de ponto flutuante de precisão dupla.

Além dos tipos, Kernighan e Ritchie (1989) incluem os qualificadores aos tipos, como o *short* e o *long*. Normalmente, com o uso do *short*, o espaço da variável, e consequentemente a faixa de valores que ela pode receber, é reduzido. Outros qualificadores são o *signed* e *unsigned*. Com o qualificador *unsigned*, um tipo somente pode receber valores positivos, o que aumenta a sua capacidade de representação de números positivos.

Na galeria de vídeos, em **Conversando sobre tipos em C**, confira um pouco mais sobre esse assunto e exemplos práticos na linguagem C.

Alocação de espaço de memória

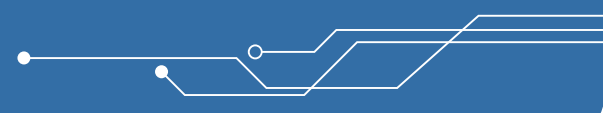
Conforme Reese (2013), um programa compilado em C trabalha com três tipos de memória:



Estática/Global: variáveis declaradas estaticamente e variáveis globais, que ficam na mesma região e são alocadas no momento que o programa inicia. Essas variáveis permanecem em memória até que o programa termine. Variáveis globais são acessíveis pelo programa como um todo e as estáticas são de acesso apenas das funções onde são definidas.

Automática: são as variáveis declaradas dentro da função e são criadas quando a função é chamada, com escopo restrito na função e tempo de vida limitado durante sua execução.

Dinâmica: com a memória alocada no *heap*, podem ser liberadas quando necessário. É um ponteiro que referencia a esse tipo de memória. Seu escopo é restrito ao escopo do ponteiro e permanece em memória até que seja liberada.



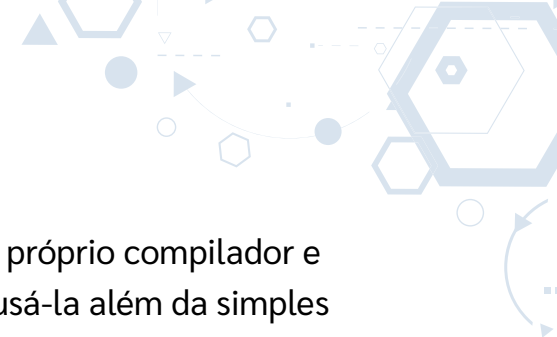
Ainda, segundo o mesmo autor, um ponteiro é uma variável que contém um endereço de outra variável, objeto ou função.

Confira a declaração das variáveis a seguir.



```
int variavel;  
int *ponteiro;
```

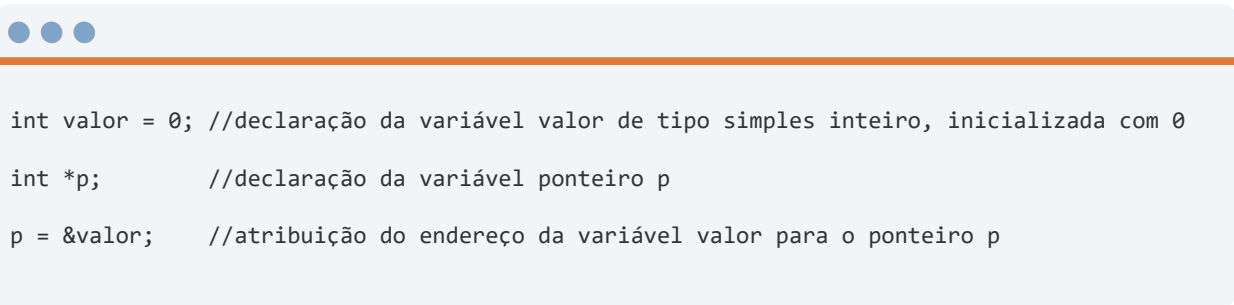
Há duas variáveis declaradas: uma de tipo simples *int* e outra é um ponteiro. Conforme cita Reese (2013), uma variável ponteiro é declarada com um tipo seguido de um asterisco e depois seu nome. Espaços em branco entre o asterisco e o nome não são considerados.



Variáveis de tipo simples têm a memória alocada pelo próprio compilador e nenhuma ação do desenvolvedor é necessária antes de usá-la além da simples declaração. Já a variável ponteiro tem alocado pelo compilador apenas espaço para receber um endereço para onde irá apontar. Ou seja, um ponteiro, em sua simples declaração, não está pronto para ser usado, primeiro ele deve receber um endereço de memória previamente alocado. Este endereço pode vir de uma variável já alocada ou pode ser realizada uma alocação de memória para que seja usada pelo ponteiro.

Referência e derreferência

Para manipular a memória, é preciso conhecer algumas funções e operadores. Conforme Reese (2013), o operador de endereço **&** é usado para obter o endereço de seu operando. Por exemplo:



```
int valor = 0; //declaração da variável valor de tipo simples inteiro, inicializada com 0
int *p;       //declaração da variável ponteiro p
p = &valor;   //atribuição do endereço da variável valor para o ponteiro p
```

Usando o operador **&** dessa forma, podemos atribuir o endereço de uma variável com memória já alocada pelo compilador ao ponteiro, fazendo com que tanto a variável **valor** quanto o ponteiro **p** apontem para o mesmo endereço de memória.

Entenda como isso ficaria na memória:

No momento das declarações, a variável **valor** poderia ser alocada pelo compilador ao endereço #100100 e a variável **p** seria alocada ao endereço #100104 (esses endereços são fictícios, não se tem autonomia para determinar o endereço que será alocado). A memória ficaria assim:



```
#100100    0
#100104    ??? (um lixo qualquer)
```

Ao fazer a atribuição da variável **valor** precedida do **&**, o que seria atribuído ao endereço da variável **p** seria o endereço de **valor**, conforme declarado a seguir:

```
#100100    0
#100104    #100100
```

Outro operador apresentado por Reese (2013) é o operador de indireção *****. Ao usar este operador com um ponteiro, o valor obtido é aquele armazenado no endereço ao qual o ponteiro aponta. Por exemplo:

```
int valor = 20; //declaração da variável valor de tipo simples inteiro, inicializada com 0
int *p = &valor; //declaração da variável ponteiro p e inicialização com o endereço de valor
printf("valor = %d\n", *p);
```

Observe que neste exemplo foi usado para imprimir a variável ***p**, e o **valor** retornado é o da variável **valor**. Isso ocorre porque **p** aponta para o endereço de **valor** e, com o uso do operador de indireção, recupera-se o valor que está nesse endereço. Veja um exemplo mais completo.

```
#include <stdio.h>          //inclusão para poder usar a função printf

int main() {

    int a;                  //Declaração de um tipo simples

    int *p;                 //Declaração de um ponteiro

    p = &a;                 //O ponteiro recebe o endereço da variável a

    a = 10;                 //Atribuimos 10 para a variável a

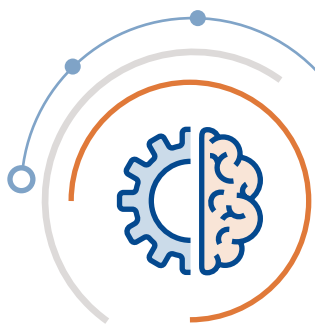
    printf("a = %d\n", *p); //Ao imprimir, usamos a indireção do ponteiro p (o valor de a, que é 10)

    *p = 20;                //Atribuimos para o local do endereço de p (a variável a) o valor 20

    printf("a = %d\n", a);  //Ao imprimir, usamos a variável a (que agora é 20)

    return 0;
}
```

O que se fez aqui foi usar um endereço já alocado para atribuir para o ponteiro, neste caso, o endereço da variável `a`. Mas, é possível alocar um espaço para ser usado pelo ponteiro. Para isso, são usadas as funções *malloc* ou *calloc*. A função *malloc* simplesmente aloca o espaço solicitado, e a função *calloc*, além de alocar o espaço, zera seu conteúdo. Como a inicialização da variável normalmente não é necessária (ela será feita em outro momento), usa-se a função *malloc*.



Mão na massa

Utilize o código apresentado para experimentar o uso de ponteiros. Crie novas variáveis, mude valores e observe o que acontece para ir se acostumando com o uso de ponteiros.

Acompanhe agora um exemplo de uso de ponteiros com a alocação de memória pela função *malloc*.

```
#include <stdio.h> //inclusão para poder usar a função printf
#include <stdlib.h> //inclusão para poder usar a função malloc

int main() {

    int a = 10; //Declarando a e inicializando com valor 10

    int *p; //Declarando o ponteiro p - neste momento o endereço que ele contém é um lixo

    p = (int *) malloc(sizeof(int)); //Alocando espaço para o valor que o ponteiro apontará,
    devolvendo o endereço alocado

    if(p == null) return 1; //Se a alocação não for possível (sem memória livre), o resultado
    da alocação é null

    *p = a; //Atribuindo o valor da variável a para o local onde p aponta

    printf("a = %d, *p = %d\n", a, *p); //Nessa impressão, a e *p possuem o mesmo valor, mas
    estão em endereços distintos

    *p = 20; //Atribuindo 20 para o local onde p aponta

    printf("a = %d, *p = %d\n", a, *p); //Nessa impressão vemos que os endereços são diferentes,
    e os valores mudaram

    free(p);

    return 0;

}
```

Observe que *p*, para receber o resultado de *malloc* precisou de um *typecast* (*int **). Isso é necessário porque o endereço retornado pelo *malloc* é do tipo *void* e o ponteiro é do tipo *int*. Outro detalhe importante é que o que foi alocado com *malloc* deve ser desalocado com *free*, quando não for mais utilizá-lo. Do contrário, aquele espaço de memória fica alocado (não pode ser usado).

Dica



Observe que, quando a alocação de memória não funciona, é retornado 1 como resultado da função *main*. Quando a aplicação chegou ao fim, é retornado 0 da função *main*. Em linguagem C, retorno 0 significa sucesso e retorno diferente de 0 é erro. Em sistemas operacionais LINUX, isso pode ser verificado facilmente. Execute um comando que dê certo e depois execute o comando `echo $?`. Você observará que o retorno será 0. Depois faça o mesmo com um comando que de erro, repita o `echo $?` e confira que o retorno será diferente de 0. Esse tipo de teste é muito usado para *scripts shell*.

Observe que, ao usar a função *malloc*, é preciso informar quantos bytes ele deve alocar. No código, foi usada outra função, *sizeof*. Isso é uma boa prática. Não se deve informar o número de bytes, porque o tamanho dos tipos varia conforme o tipo da máquina hospedeira. Assim, sempre que se quer informar o espaço a ser alocado, executa-se a função *sizeof*, passando como parâmetro o tipo desejado, e o sistema aloca o espaço necessário para armazenar um valor daquele tipo.

Na galeria de vídeos, em **Manipulando memória com C**, confira exemplos práticos na linguagem C.

Passagem de parâmetros em funções

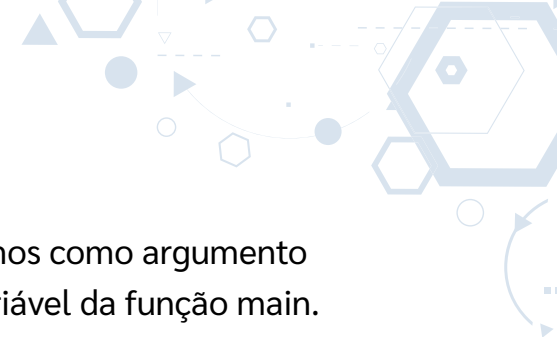
Quando se chama uma função, podem ser passados parâmetros. Conforme Kernighan e Ritchie (1989), em C todos os argumentos são passados “por valor”. Isso significa que é feita uma cópia do valor e é sobre a cópia que são realizadas as operações da função, então a variável original não é alterada. Quando se precisa que a variável passada como argumento seja alterada pela função chamada, é necessário passar o endereço da variável, ao invés da variável. Com isso, a função irá copiar o endereço e manipular o que está no endereço, o que altera a variável desejada. Veja um exemplo.

```
#include <stdio.h>

void adicionalPorValor(int valor) {
    valor++;
    printf("Argumento valor = %d\n", valor);
}

void adicionalPorReferencia(int *valor) {
    (*valor)++;
    printf("Argumento valor = %d\n", *valor);
}

int main() {
    int valor_original = 100;
    adicionalPorValor(valor_original);
    printf("Valor_original = %d\n\n", valor_original);
    adicionalPorReferencia(&valor_original);
    printf("Valor_original = %d\n", valor_original);
    return 0;
}
```



Podemos conferir, neste exemplo, que quando passamos como argumento a variável, a operação de adicionar não é refletida na variável da função main. Quando no argumento é passado o endereço da variável, a operação realizada na função reflete-se na variável da função main.

Tipos homogêneos

Tipos homogêneos (*arrays*) são a representação de locais contíguos de memória, permitindo definir uma determinada quantidade de valores de mesmo tipo. Backes (2013) descreve *array* como “um conjunto de variáveis de um mesmo tipo, com a vantagem de estarem todas associadas ao mesmo nome e igualmente acessíveis por um índice.”

Deitel e Deitel (2011) explicam que, em linguagem C, um *array* possui um nome, e para acessar um de seus elementos usa-se o operador [] (abre e fecha colchetes). O seu primeiro elemento possui índice 0 (zero). Deitel e Deitel (2011) explicam que o índice que fica entre os colchetes é chamado formalmente de *subscrito*. Confira uma declaração de um vetor em C.



```
int vetor[20];
```

Com esta declaração, o compilador irá alocar 20 espaços de inteiro para a variável vetor. Para obter um resultado semelhante usando ponteiros, é feito o seguinte:

```
int *p;
p = (int *) malloc(sizeof(int) * 20);
```

Com o malloc alocando 20 vezes o espaço de inteiro temos o mesmo espaço alocado que para o vetor.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int vetor[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int *ponteiro = (int *) malloc(sizeof(int) * 10);
    int i;
    for(i = 0; i < 10; i++) {
        ponteiro[i] = vetor[i] * 10;
        printf("vetor[%d] = %d, ponteiro[%d] = %d\n", i, vetor[i], i,
ponteiro[i]);
    }
    free(ponteiro);
    return 0;
}
```

Na declaração de um vetor, é possível inicializá-lo com o valor de cada elemento separado por vírgulas. Isso só pode ser feito na inicialização. Dentro do laço *for*, atribui-se a cada local endereçado pelo ponteiro o valor da mesma posição de vetor multiplicado por 10. A impressão do resultado mostra o que foi obtido.

Você pode observar que é possível usar o operador `[]` em ponteiro da mesma forma que se usa no vetor – na verdade, o vetor é um ponteiro, só que ele é alocado pelo compilador.

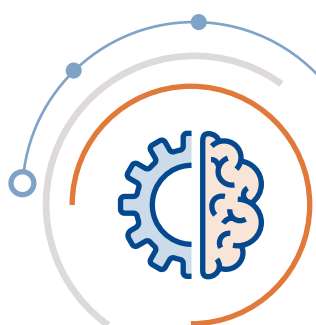
Um ponteiro pode ter seu endereço alterado por operações aritméticas. Por exemplo, se um ponteiro é do tipo *char* (ocupa 2 bytes), e seu endereço atual for 200 e se somado 2 a ele, ele terá como novo endereço 204. Isso acontece porque na soma de endereços o tamanho do tipo é considerado, ou seja, na soma ele fará $200 + (2 * 2)$. Confira um exemplo do que é possível se fazer com a aritmética de ponteiros.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int vetor[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int *ponteiro = (int *) malloc(sizeof(int) * 10);
    int i;
    for(i = 0; i < 10; i++) {
        *ponteiro = vetor[i] * 10;
        printf("vetor[%d] = %d, ponteiro[%d] = %d\n", i, vetor[i], i,
*ponteiro);
        ponteiro++;
    }
    ponteiro -= 10;
    free(ponteiro);
    return 0;
}
```

Neste exemplo, ao invés de usar o operador [], usou-se a indireção para obter o valor para onde o ponteiro aponta e para atribuir o valor para onde o ponteiro aponta. O resultado obtido é o mesmo. Foi usada a aritmética de ponteiros para ir avançando para a posição da memória que se quer armazenar os valores. Para poder liberar a memória de ponteiro, é preciso retornar à sua posição inicial (`ponteiro -= 10;`).

Quando se usa um vetor por simples declaração, não é possível aplicar a aritmética de ponteiros sobre ele, pois seu valor é constante (não permite modificação). Ou seja, se você fizer `vetor++`, obterá um erro do compilador. Com isso, pode-se, agora, dizer as diferenças entre declarar um vetor e definir um ponteiro com espaço para vários elementos. Um vetor tem seu endereço inicial constante e não precisa ser desalocado ao final, o que é responsabilidade do compilador. Um ponteiro precisa ser alocado antes de ser usado, permite aritmética de ponteiros e precisa ser desalocado quando não for mais ser usado.



Mão na massa

Repita os códigos apresentados aqui, faça alterações nos tamanhos de alocação e no tamanho do vetor. Confira seu comportamento para ir ganhando intimidade com o uso de ponteiros.



Curiosidade

Dado a característica dos arrays de serem alocações de memória contígua, eles são as estruturas mais performáticas de busca de posição, pois isso ocorre por um simples cálculo aritmético, indo direto ao endereço para recuperar o valor desejado daquela posição.

Um tipo homogêneo especial é a *string*. A linguagem C não possui *strings* como tipos simples. Para usar *strings* em C, são criados vetores de *char*. Outro aspecto peculiar da linguagem C para identificar se uma *string* acabou é verificado se foi alcançado o caractere `\0` (barra zero). Sempre que você for usar uma *string* deve terminá-la com um `\0`. Ou seja, se sua *string* terá tamanho 10, o vetor deve ser capaz de suportar 11 caracteres, os 10 de sua *string* mais o `\0`. Há uma biblioteca do C que facilita o uso de *strings*, conforme apresentado no exemplo a seguir.

```

#include <stdio.h>
#include <strings.h> //inclusão para poder usar strcmp
int main() {
    char nome_1[] = "Jose Antonio";
    char nome_2[] = {'J', 'o', 's', 'e', ' ', 'A', 'n', 't', 'o', 'n', 'i', 'o', '\0'};
    char nome_3[50];
    int i = 0;
    for(i = 0; i < 12; i++) {
        nome_3[i] = nome_1[i];
    }
    nome_3[12] = '\0';
    int dif;
    dif = strcmp(nome_1, nome_2);
    if(dif == 0) printf("nome_1 e nome_2 são iguais");
    dif = strcmp(nome_2, nome_3);
    if(dif == 0) printf("nome_2 e nome_3 são iguais");
    return 0;
}

```

A biblioteca para manipulação de *strings* em C em alguns compiladores é de *string.h*; em outros, é *strings.h*. Ao compilar, se der erro nessa biblioteca, é só trocar uma pela outra. Perceba que, neste exemplo, é possível inicializar uma *string* de muitas formas. Na declaração, é possível passar o texto entre aspas duplas para o vetor. Não se coloca o tamanho do vetor para deixar o compilador calcular conforme o tamanho que for inicializado. Assim, neste caso, o vetor *nome_1* terá o tamanho 13, sendo 12 para os caracteres do nome passado mais um caractere `\0`. Quando se passa uma *string* entre aspas duplas, o compilador coloca o `\0` automaticamente. Esse tipo de atribuição só é possível na declaração.

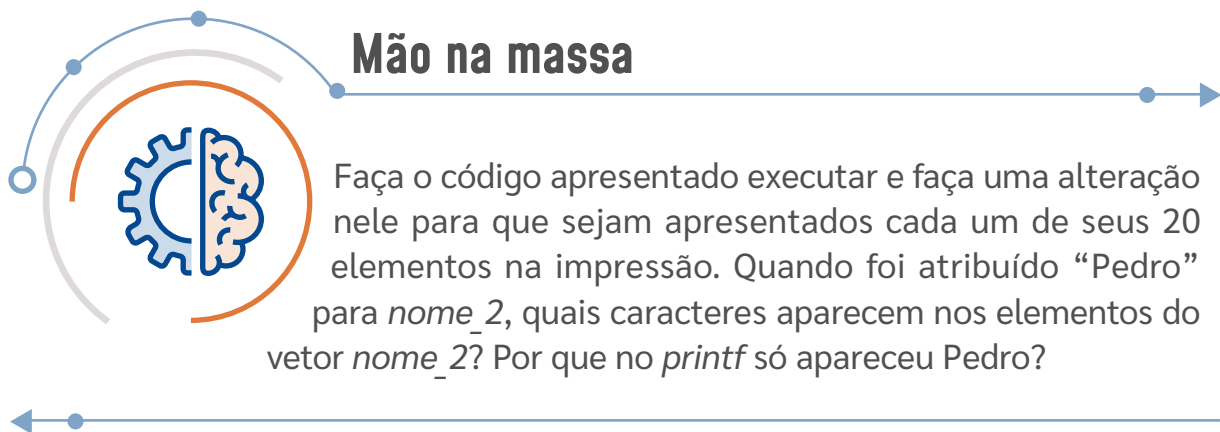
A forma de inicialização usada em *nome_2* é a mesma que foi usada com o vetor de inteiros, mas agora passando os *chars* da *string* desejada e o `\0`. Em *nome_3*, usou-se um laço para ir inserindo caractere a caractere na *string* e, depois do laço, é incluído o `\0` na última posição. Nesse caso, *nome_3* tem 50 posições, mas só ocupa 13.

A função *strcmp* permite comparar *strings*. Se a primeira for maior que a segunda, ela retorna um número positivo; se a segunda for maior que a primeira, ela retorna um número negativo. Se a primeira e a segunda forem iguais, ela retorna zero. Observe que, independentemente do tamanho do vetor, as *strings* foram consideradas iguais.

Outra função muito usada com *strings* é *strcpy*. Com essa função, é possível fazer cópias de *string*, mas observe que a função não verifica se a *string* destino tem tamanho suficiente para receber a *string* de origem (incluindo o `\0`). E, se não tiver, ocorrerá um erro. O primeiro parâmetro é a *string* destino e o segundo a *string* origem. Confira o exemplo de uso.

```
#include <stdio.h>
#include <strings.h> //inclusão para poder usar strcpy
int main() {
    char nome_1[] = "Alberto Roberto";
    char nome_2[20];
    strcpy(nome_2, nome_1);
    printf("%s \n%s\n", nome_1, nome_2);
    strcpy(nome_2, "Pedro");
    printf("%s \n%s\n", nome_1, nome_2);
    return 0;
}
```

Neste exemplo, usou-se o *strcpy* para copiar o conteúdo de *nome_1* em *nome_2*. Foi observado o cuidado de fazer com que *nome_2* tivesse espaço para receber todos os caracteres de *nome_1* mais o caractere `\0`. No segundo uso de *strcpy*, foi realizada uma cópia de uma *string* entre aspas duplas para *nome_2*. Sempre que é usada uma *string* entre aspas duplas o compilador coloca o `\0`.



Na galeria de vídeos, em **Tipos homogêneos e strings**, confira um pouco mais sobre esse assunto e acompanhe exemplos práticos na linguagem C.

Tipos heterogêneos

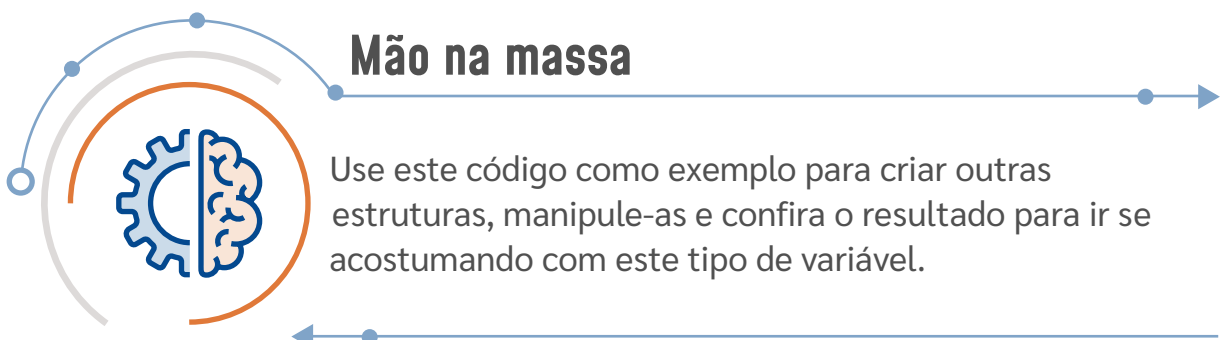
Nem sempre os dados que se quer relacionar são de mesmo tipo, como observado em tipos homogêneos. Por exemplo, é possível querer relacionar os dados de uma data com um inteiro para o dia, o mês, como uma *string* e outro inteiro para o ano (13, março, 1930). Um vetor não poderia manter estes três valores. Para isso, são necessários tipos heterogêneos. Para criar um tipo heterogêneo em C, tem-se o *struct*.

Conforme Kernighan e Ritchie (1989), um *struct* é uma coleção de variáveis que podem ser de tipos diferentes. Para armazenar, por exemplo, o nome, CPF e salário em uma variável, uma solução é o uso de um *struct*. Veja um exemplo de uso de *struct*.

```
#include <stdio.h>

int main() {
    struct empregado {
        char nome[50];
        double cpf;
        double salario;
    };
    struct empregado e;
    strcpy(e.nome, "Maria Augusta");
    e.cpf = 12345678912;
    e.salario = 5000;
    printf("Nome: %s\nCPF: %.0f\nSalario: %.2f", e.nome, e.cpf,
e.salario);
    return 0;
}
```

Neste exemplo, foi criada uma *struct* empregado. Para usar esta estrutura, é preciso definir uma variável. Aqui foi usada a variável *e*, e seu tipo será definido com a palavra *struct*, seguida do nome da estrutura criada no início do programa. Depois de declarada a variável, ela já pode ser usada (o compilador já aloca espaço para a estrutura). Para acessar cada um dos membros da estrutura, é usado o operador ponto (variável.membro).



Um ponteiro pode apontar para um espaço alocado para uma estrutura. Quando a estrutura está sendo acessada por um ponteiro, o operador usado para acessar os membros é o `->` (chamado seta). Você poderá observar mais de seu uso em tipos abstratos de dados.

Na galeria de podcasts, em **Tipos heterogêneos**, será discutido um código na linguagem C.

TIPO ABSTRATO DE DADOS [TAD]

Para usar algo no computador, é necessária uma abstração para tornar viável seu uso computacional. Moraes (2001) explica que um tipo abstrato de dados é um conjunto de valores associados às operações que podem ser realizadas sobre esses valores. Essa explicação fica mais clara entendendo tipo de dados e estrutura de dados.

Uma definição formal para tipo de dados é oferecida por Backes (2016): “Um **tipo de dado** define o conjunto de **valores** (domínio) e **operações** que uma variável pode assumir.” Outra definição importante oferecida pelo autor é a de estrutura de dados: “Uma **estrutura de dados** consiste em um conjunto de tipos de dados em que existe algum tipo de relacionamento lógico estrutural.”

Juntando estas duas definições, tem-se o tipo abstrato de dados, que Backes (2016) define como: “[...] um **tipo abstrato de dados**, ou **TAD**, é um conjunto de dados estruturados e as operações que podem ser executadas sobre esses dados.”

A construção do tipo abstrato de dados, conforme Backes (2016), é feita a partir de tipos simples, como *char*, *int*, *double* ou *float*, ou mesmo tipos estruturados (*structs*). Uma vez estabelecida a estrutura dos dados, são criadas as operações que podem ser realizadas sobre esta estrutura.

Dentre os benefícios de uso de um TAD, Backes (2016) apresenta a possibilidade de ocultar os dados da estrutura (proteção), e qualquer operação que seja realizada sobre esses dados será feita por meio de uma interface que oferece operações específicas.

Confira um exemplo com a estrutura de empregado. Neste exemplo, são necessários três arquivos: um para o TAD, um para o cabeçalho do TAD e outro que usará o TAD. Começa-se pelo TAD.

```
#include <strings.h>
#include <stdlib.h>
#define TAMANHONOME 50
struct empregado {
    char nome[TAMANHONOME];
    double cpf;
    double salario;
};
struct empregado* get_empregado() {
    return (struct empregado *) malloc(sizeof(struct empregado));
}
void set_nome(struct empregado *e, char *nome) {
    int tamanho = strlen(nome);
    strcpy(e->nome, nome);
    if(tamanho > TAMANHONOME) {
        nome[TAMANHONOME - 1] = '\0';
    }
}
char* get_nome(struct empregado *e) {
    return e->nome;
}
void set_cpf(struct empregado *e, double cpf) {
    e->cpf = cpf;
}
double get_cpf(struct empregado *e) {
    return e->cpf;
}
```

```
void set_salario(struct empregado *e, double salario) {
    e->salario = salario;
}
double get_salario(struct empregado *e) {
    return e->salario;
}
```

O arquivo com o TAD contém a definição da estrutura de dados que será usada e todas as funções que sabem manipular este tipo de estrutura. Observe que, como a definição da estrutura está neste arquivo, quem for usar este TAD não pode criar uma variável por simples declaração, pois o compilador não saberá o tamanho da variável. Nesse caso, será necessário criar um ponteiro. O ponteiro tem um tamanho fixo, independentemente do tipo de estrutura que ele aponta, e o compilador sabe alocar espaço para o endereço do ponteiro. Outro ponto a observar é a função *get_empregado*. Essa função é necessária, porque quem for usar este TAD não consegue fazer o *malloc* para reservar o espaço, pois o tamanho da estrutura não é conhecido. Por essa razão, o TAD tem que devolver um espaço já alocado para quem for usá-lo. Finalmente, é necessário que sejam criadas funções para devolver o valor dos membros, pois como a estrutura não é conhecida, uma tentativa de acesso a um membro desta estrutura apresentará erro de compilação. Agora será estudado o arquivo de cabeçalho do TAD.

```

struct empregado;
struct empregado* get_empregado();
void set_nome(struct empregado *e, char *c);
char* get_nome(struct empregado *e);
void set_cpf(struct empregado *e, double cpf);
double get_cpf(struct empregado *e);
void set_salario(struct empregado *e, double salario);
double get_salario(struct empregado *e);

```

O cabeçalho tem como objetivo informar ao compilador o que ele deve validar (verificar se a codificação está correta). Assim, ele cita a definição de um *struct* como existente e apresenta todas as assinaturas das funções. Finalmente confira um programa que irá usar o TAD.

```

#include <stdio.h>
#include "empregado.h"
int main() {
    struct empregado *e = get_empregado();
    set_nome(e, "Jurandir Matias de Souza Prado");
    set_cpf(e, 98765432100);
    set_salario(e, 3500);
    printf("%s\nCPF %.0f\nSalario: R$ %.2f\n", get_nome(e), get_cpf(e), get_salario(e));
}

```

Observe que, para quem usa o TAD, o que ele precisa é incluir o cabeçalho do TAD e usar as funções que ele fornece.

A compilação do TAD deve ser feita da seguinte forma:

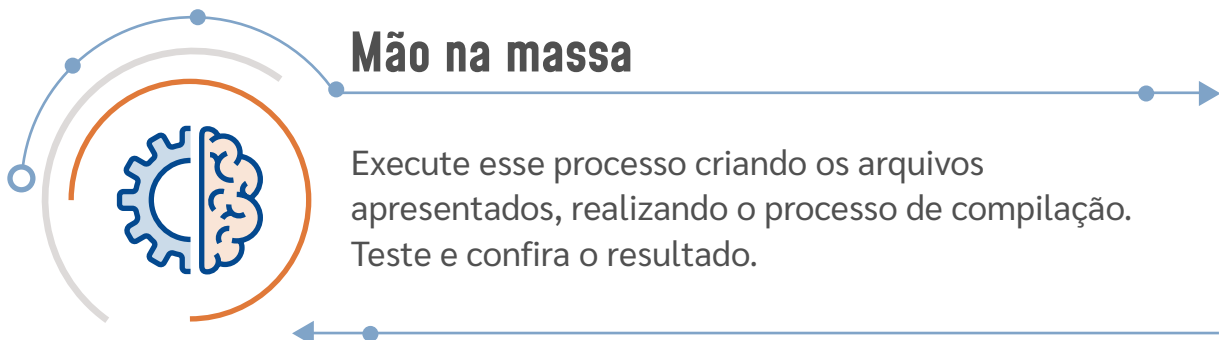
```
gcc -c tad.c
```

Admitindo que o nome do arquivo com o código do TAD é `tad.c`, esse é o comando de compilação. Será gerado um arquivo `tad.o`. Este arquivo não é executável, pois é um arquivo de biblioteca. O arquivo de cabeçalho não é compilado.

Para compilar a aplicação que usa o TAD, usa-se o seguinte comando:

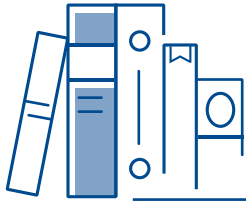
```
gcc aplicação.c tad.o -o aplicação
```

Admitindo que o nome do arquivo que contém o código da aplicação é `aplicacao.c`, este seria o comando para compilá-lo para gerar o arquivo `aplicacao.exe` ao final do processo (no LINUX, o nome ficaria sem extensão).



Na galeria de vídeos, em **Criando bibliotecas**, confira um pouco mais sobre esse assunto e conheça exemplos práticos na linguagem C.

Você estudou os elementos mais importantes de suporte de estruturas de dados, os tipos homogêneos e heterogêneos. Observou a forma como as funções fazem para passar parâmetros por valor e por referência, conhecimento importante quando são criadas bibliotecas. E, finalmente pôde compreender a definição de tipos abstratos de dados, bem como trabalhar a criação de um tipo abstrato de dados como exemplo.



REFERÊNCIAS

MORAES, C. R. **Estruturas de dados e algoritmos: uma abordagem didática.** São Paulo: Berkeley Brasil, 2001.

LONDON, K. **Dominando algoritmos com C.** Rio de Janeiro: Editora Ciência Moderna Ltda., 2000.

KERNIGHAN, B. W.; RITCHIE, D. M. C, **a linguagem de programação: padrão ansi.** 2. ed. Rio de Janeiro: Campus, 1989.

BACKES, A. **Linguagem C: completa e descomplicada.** Rio de Janeiro: Elsevier, 2013.

DEITEL, P.; DEITEL, H. **Como programar em C.** 6. ed. São Paulo: Pearson Prentice Hall, 2011.

REESE, R. **Understanding and Using C Pointers.** Gravenstein Highway North: O'Reilly Books, 2013.

BACKES, A. R. **Estrutura de dados descomplicada: em linguagem c.** Rio de Janeiro: Elsevier, 2016.



SENAI



Videoflix



A seguir, confira uma série de vídeos para demonstrar alguns conhecimentos que são mais fáceis de se entender de forma visual. Você estudará o uso do compilador, manipulação de memória com a linguagem C, manipulação de tipos homogêneos e strings e a criação de bibliotecas, parte bem importante na construção de tipos abstratos de dados. Vamos lá!



O compilador C



[https://player.vimeo.com/
video/800842054?h=e374cd4ff2](https://player.vimeo.com/video/800842054?h=e374cd4ff2)



Comparando linguagem de alto nível e baixo nível



[https://player.vimeo.com/
video/800842078?h=bf180b0596](https://player.vimeo.com/video/800842078?h=bf180b0596)



Conversando sobre tipos em C



[https://player.vimeo.com/
video/800842115?h=36a17804f5](https://player.vimeo.com/video/800842115?h=36a17804f5)



Manipulando memória com C



[https://player.vimeo.com/
video/800842137?h=fd7c62d960](https://player.vimeo.com/video/800842137?h=fd7c62d960)



Tipos homogêneos e strings



[https://player.vimeo.com/
video/800842154?h=3146a7537c](https://player.vimeo.com/video/800842154?h=3146a7537c)



Criando bibliotecas

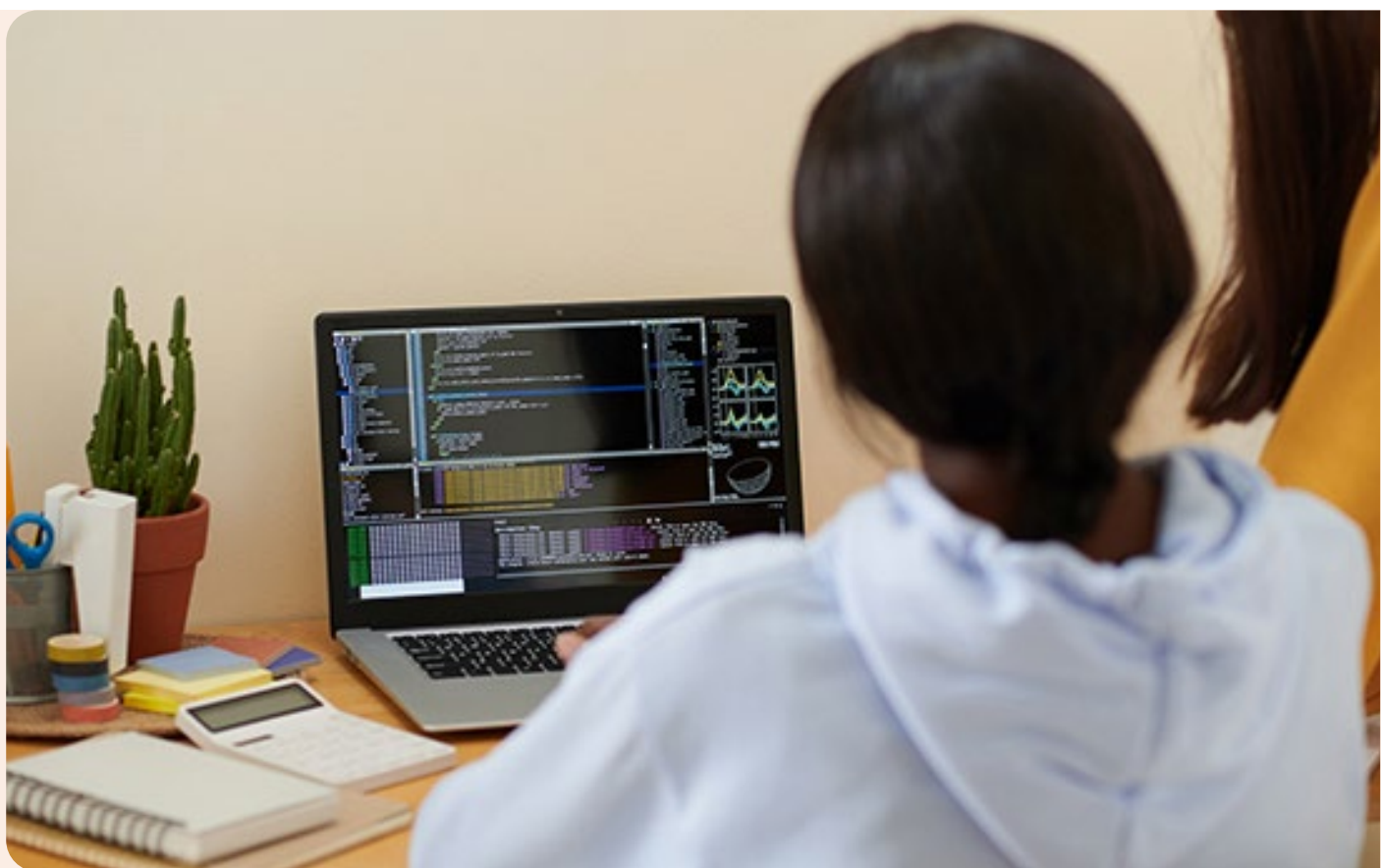


[https://player.vimeo.com/
video/800842174?h=ced8cf256b](https://player.vimeo.com/video/800842174?h=ced8cf256b)



Infocast

A seguir, comece a compreender e a construir códigos com linguagem C. Inicialmente, você estudará os tipos simples e ponteiros em um código de exemplo, o uso de tipos heterogêneos, o uso da função `printf` e da manipulação de *strings*. Essa galeria de podcasts facilitará seu entendimento na construção dos exercícios propostos, além de prepará-lo para a escrita de códigos.





Tipos simples e ponteiros

Olá, estudante!

Confira a imagem que irá te auxiliar nesse podcast.



```
#include <stdlib.h>

int main() {
    int var1;
    int var2[2];
    int *var3;
    int *var4;

    var1 = 10;
    var3 = &var1;
    *var3 = 20;
    var3 = (int *) malloc(sizeof(int));
    *var3 = 50;

    var2[0] = 1;
    var2[1] = 2;
    var4 = var2;
    *var4 = 11;
    var4[1] = 22;
    var4 = (int *) malloc(sizeof(int) * 2);
    *var4 = 111;
    var4++;
    *var4 = 222;
}
```



```
*ponteiro_empregado = (struct empregado *) malloc(sizeof(struct
    empregado));

strcpy(ponteiro_empregado->nome, "Alberto Carneiro");
ponteiro_empregado->tempo_servico_anos = 11;
novo_empregado.salario_base = 8300.00;
}
```

Vamos falar sobre tipos na linguagem C, tratando especificamente da diferença entre tipos simples e ponteiros.

Começamos a diferenciar tipos simples pela sua declaração. Um tipo simples é declarado indicando seu tipo e o nome da variável. Essas informações são suficientes para que o compilador saiba como alocar espaço para ela. O desenvolvedor, após declará-la, pode usá-la, pois é o compilador que se preocupa com a alocação de espaço para ela. Você pode observar isso em *var1*.

Um ponteiro tem a sua declaração um pouco diferente. Para declarar um vetor, informamos seu tipo, que, na verdade, é o tipo do valor que ele apontará e não o tipo do ponteiro. O tipo real do ponteiro é ponteiro. Uma vez declarado, o ponteiro pode ser usado apenas para receber um endereço para onde ele vai apontar. Antes que ele receba esse endereço, não pode receber atribuições por indireção. Temos o ponteiro *var3*, que podemos ver como foi declarado. Vemos que *var3* recebe o endereço de *var1* usando o operador de derreferência & com *var1*. Usando *var1*, o que recebemos é o valor que lhe foi atribuído; usando *&var1*, obteremos o endereço de *var1*.

Uma vez que o endereço de *var1* foi atribuído para *var3*, podemos atribuir um novo valor ao endereço que ele aponta. Para isso, usamos o operador de indireção *. Colocando o operador de indireção em *var3*, quando atribuímos um valor a *var3*, esse valor é armazenado na posição de memória. A partir do momento que atribuímos o valor por indireção por *var3*, tanto *var1* quanto **var3* vão devolver o mesmo valor, pois apontam para o mesmo endereço.

Se quisermos que *var3* aponte para um endereço livre e possa ser usado como uma variável independente, usamos a alocação de memória - neste caso usamos o *malloc*. O *malloc* entrega um endereço livre com espaço suficiente, conforme solicitado pelo argumento, para armazenar o que for solicitado.

Podemos ter armazenamentos de vários elementos usando tipos homogêneos - vetores. A declaração de *var2* exemplifica isso. A sua declaração é simplesmente indicar o tipo, o nome da variável e a quantidade de elementos que ela deva suportar entre colchetes. Com essas informações, o compilador é capaz de alocar espaço para ela. O desenvolvedor pode utilizá-la de imediato, atribuindo valores para cada endereço alocado. Para acessar cada item do vetor, usamos o operador de colchete. O endereço inicial é acessado pelo 0 entre colchetes; e o próximo é o 1, e assim por diante.

Quando usamos ponteiros, também podemos alocar espaço para mais de um elemento. Temos o endereço de *var2*, que tem espaço alocado para dois inteiros. Podemos atribuir o endereço de *var2* para *var4*. Confira, no exemplo, que não usamos o operador de referência & nesse caso. Quando declaramos um vetor, se usarmos o nome da variável sem o operador de colchetes, o que ele devolve é seu endereço inicial. O mesmo resultado seria obtido se usássemos *&var2[0]*. Observe que, para atribuir valor para os endereços alocados para *var2* usando *var4*, podemos usar a indireção ou o operador de colchetes, conforme o exemplo.

Da mesma forma que fizemos com um ponteiro, podemos atribuir um espaço livre de memória do tamanho desejado para o ponteiro usando *malloc*. Depois de alocado um espaço, os endereços não são mais compartilhados. Podemos usar operador de indireção ou de colchetes para atribuir valores ao espaço alocado. No exemplo, estamos usando indireção e avançamos o endereço do ponteiro por aritmética de ponteiros.

Experimente usar este exemplo para fixar o entendimento destas manipulações. Use comandos de impressão para ver os resultados que estão sendo alcançados.

Espero que tenham gostado.

Até mais!



Tipos heterogêneos

Olá, estudante!

Confira a imagem que irá te auxiliar nesse podcast.



```
#include <stdlib.h>
#include <strings.h>
int main() {
    struct empregado {
        char nome[50];
        int tempo_servico_anos;
        double salario_base;
    };

    struct empregado novo_empregado;
    strcpy(novo_empregado.nome, "Joaquim da Silva Sauro");
    novo_empregado.tempo_servico_anos = 0;
    novo_empregado.salario_base = 3500.00;

    struct empregado *ponteiro_empregado;
    ponteiro_empregado = &novo_empregado;

    strcpy(ponteiro_empregado->nome, "Joao Gilberto Gomes");
    ponteiro_empregado->tempo_servico_anos = 5;
    novo_empregado.salario_base = 5000.00;
```



```
*ponteiro_empregado = (struct empregado *) malloc(sizeof(struct
    empregado));

strcpy(ponteiro_empregado->nome, "Alberto Carneiro");
ponteiro_empregado->tempo_servico_anos = 11;
novo_empregado.salario_base = 8300.00;
}
```

Vamos falar sobre registros ou tipos heterogêneos na linguagem C. Acompanhe a definição, sua declaração e uso com alocação pelo compilador e alocação pelo desenvolvedor.

A definição de um tipo estruturado inicia com a palavra reservada *struct*, seguida do nome da estrutura, conforme podemos ver no exemplo. Iniciamos um bloco entre chaves para determinar a estrutura interna do tipo heterogêneo, podendo ter declarações de quaisquer tipos. Nesse exemplo, usamos um vetor de *char*, um inteiro e um *double*. Encerramos a definição com um ponto e vírgula após a chave. Com a definição, nada foi alocado de memória. Ela serve apenas para o compilador saber quanto alocar se alguma variável desse tipo for declarada.

Em seguida, temos a declaração de uma variável desse tipo. Agora o compilador aloca espaço suficiente para armazenar todos os elementos internos da estrutura. Uma vez declarada, está pronta para ser usada. É só atribuir os valores para seus elementos internos. Isso é feito com o operador de ponto. Indicamos o nome da variável, separamos com o (ponto) e indicamos o nome do elemento interno ao qual vamos realizar a operação.

No primeiro caso, foi feita uma cópia de string para o primeiro campo, que é um vetor de *char*. Em seguida, atribuímos um valor para o segundo campo e depois para o terceiro.

Podemos usar ponteiros sobre esses tipos estruturados. Declaramos o ponteiro citando o tipo definido e já temos um ponteiro que pode receber um endereço desse tipo. O endereço pode ser de algo criado pelo compilador,

como o de *novo_empregado*. Pegamos o endereço de *novo_empregado* com o operador de referência & e atribuímos ao *ponteiro_empregado*.

Com o endereço definido, pode-se usar o ponteiro para redefinir os valores dos campos internos. Observe que agora usamos o operador seta e não o ponto.

Todo o restante permanece com as mesmas características.

Se quisermos atribuir um endereço livre para *ponteiro_empregado*, usamos o *malloc*. Com um endereço livre atribuído, se definirmos valores para seus campos, eles não afetarão mais a variável *novo_empregado*.

Experimente usar este exemplo para fixar o entendimento destas manipulações. Use comandos de impressão para ver os resultados que estão sendo alcançados.

Com esse exemplo, fica mais claro o uso de tipos heterogêneos. Espero que tenha gostado.

Até mais!



Comandos de manipulação de *strings* e o printf

Olá, estudante!

Confira a imagem que irá te auxiliar nesse podcast.



```
#include <stdio.h>
printf("%s", variavel);
printf("%20s", variavel);
printf("%-20s", variavel);
printf("%d", variavel);
printf("%20d", variavel);
printf("%-20d", variavel);
printf("%020d", variavel);
printf("%f", variavel);
printf("%10.2f", variavel);
printf("%-20s: %15.2f\n", titulo, valor);
```

```
#include <strings.h>
strcpy(destino, origem);
strncpy(destino, origem, n);
strcat(destino, origem);
strncat(destino, origem, n);
strlen(str);
strcmp(str1, str2);
```

Vamos conhecer um pouco melhor a função `printf`, muito usada nas aplicações, pois sempre temos que imprimir resultados ou quaisquer outras coisas que a aplicação queira apresentar ao usuário. Também vamos conhecer algumas funções da biblioteca *strings*, muito usada na manipulação de *strings*.

Para usar a função `printf`, o arquivo do código-fonte deve incluir o arquivo de cabeçalho `stdio.h`. O primeiro parâmetro da função é uma *string* de formatação. Nessa *string*, pode conter qualquer texto, mas existem alguns elementos que possuem significado especial. São as sequências de conversão, que iniciam com `%`. O número de parâmetros da função vai depender da quantidade de sequências de conversão. O primeiro parâmetro é a *string* formatadora, e os demais são as variáveis que serão usadas nas *strings* de conversão na ordem em que elas aparecem na *string* formatadora.

Os três primeiros exemplos da função `printf` usam a sequência de conversão. Emprega-se este formatador para indicar para a função que naquela posição deve ser inserida a variável, que deve ser um vetor de *char* terminando com `'\0'`. Temos três variações no uso do `%s`:

1. Somente o `%s`, o que faz com que a variável seja inserida na posição da *string* formatadora.
2. Antes do `s` é incluído um tamanho. Neste caso, a *string* inserida ocupará um espaço de 20 caracteres e ficará alinhada à direita.
3. Antes do `s` é incluído um tamanho negativo. Neste caso, a *string* ocupará um espaço de 20 caracteres e ficará alinhada à esquerda.

Os quatro exemplos a seguir mostram a formatação com inteiros. Temos quatro variações apresentadas:

1. Somente o `%d`, que faz com que o valor em variável seja inserido naquele ponto.
2. Antes do `d` temos um tamanho. Neste caso, o valor da variável ocupará um espaço de 20 caracteres, ficando alinhado à direita.
3. Antes do `d` temos um tamanho negativo. Neste caso, o valor da variável ocupará um espaço de 20 caracteres, ficando alinhado à esquerda.
4. Antes do `d` temos um tamanho iniciado por zero. Neste caso, o valor da variável ocupará um espaço de 20 caracteres, sendo preenchida com zeros à esquerda.

Depois temos dois exemplos com valores de ponto flutuante. Há duas variações apresentadas:

1. Somente o %f, que faz com que o valor seja inserido naquela posição.
2. Antes do f, um tamanho, um ponto e outro tamanho. O primeiro tamanho é o espaço total que será ocupado pelo número de ponto flutuante; o segundo tamanho é a quantidade de casas decimais que será usada na apresentação. O tamanho total inclui o ponto e as casas decimais.

Um exemplo mais genérico para ficar claro a forma de formatar uma apresentação tem a seguinte característica:

A primeira parte é uma *string* de tamanho 20 alinhada à esquerda, seguido de dois pontos e depois um valor de ponto flutuante com tamanho total 15 e com duas casas decimais.

Isso resume o uso da função printf. Veja agora a biblioteca *strings*.

A primeira função é *strcpy*, que tem dois parâmetros. O primeiro parâmetro é o vetor *char*, que receberá a cópia; o segundo é o vetor *char* copiado.

A segunda função é *strncpy*, que tem três parâmetros. O primeiro parâmetro é o vetor *char*, que receberá a cópia; o segundo é o vetor *char* copiado; e o terceiro, a quantidade de caracteres que serão copiados.

A terceira função é *strcat*, que tem dois parâmetros. O primeiro parâmetro é o vetor *char*, que receberá a concatenação; o segundo é o vetor *char* copiado na concatenação.

A quarta função é *strncat*, que tem três parâmetros. O primeiro parâmetro é o vetor *char*, que receberá a concatenação; o segundo é o vetor *char* copiado na concatenação; e o terceiro, a quantidade de caracteres que serão concatenados.

A quinta função é *strlen*, que possui um parâmetro. O parâmetro é o vetor de *char*, que será usado para identificar o tamanho da *string*. Para identificar o tamanho da *string*, ele conta os caracteres até encontrar o caractere '\0'. Logo, o tamanho não precisa coincidir com o tamanho do vetor.

A sexta função é *strcmp*, que tem dois parâmetros. A *string* no primeiro parâmetro for menor que a *string* do segundo parâmetro, o valor retornado será um número negativo. Se a *string* no primeiro parâmetro for maior que a *string* do segundo parâmetro, o valor retornado será positivo. Se forem iguais, o valor retornado é zero.

Esse foi um resumo das principais funções da biblioteca *strings*. Espero que você tenham gostado. Até mais!



Quero saber +

Gostou do assunto? Você pode aprender ainda mais sobre linguagens e módulos, buscando novos horizontes, sites, links, aplicativos e livros. Saiba mais lendo e conferindo os materiais a seguir.



Compiladores

Se você quer conhecer melhor os compiladores, seus aspectos de construção e uso, este site irá ajudá-lo bastante. Clique no ícone e confira.



<https://johnidm.gitbooks.io/compiladores-para-humanos/content/>



Biblioteca stdio

E, para se aprofundar em uma das bibliotecas mais usadas no C, o site a seguir tem referências explicando as funções disponíveis por ela.



<https://petbcc.ufscar.br/stdio/>



Biblioteca *stdlib*

Outra biblioteca muito bacana você confere clicando no ícone, a seguir. Aprofunde seus conhecimentos em *stdlib*. Confira!



<https://petbcc.ufscar.br/stdlib/>



Visual Studio Code

Aprenda a programar usando a ferramenta a seguir.



<https://code.visualstudio.com/>



Instalação GCC

Clique no ícone, a seguir, e confira uma explicação textual sobre como instalar o GCC.



<https://www.mundodoshackers.com.br/como-instalar-o-gcc-no-windows>





Tutorial instalação GCC

A seguir, assista a um vídeo no youtube sobre a instalação do GCC. Clique e confira.



<https://www.youtube.com/watch?v=FzPBZjkoEmA>



Resumindo

Agora chegou o momento de conferir um resumo dos principais conhecimentos apresentados ao longo de seus estudos até aqui. Este resumo foi elaborado em formato de checklist, para que você possa assinalar os itens que considera já ter desenvolvido e, caso sinta a necessidade, retome os estudos. Aproveite mais esta oportunidade de construção de saberes.

- ☐ Entendi a diferença de linguagens interpretadas, híbridas e compiladas.
- ☐ Construí códigos básicos da linguagem C.
- ☐ Aprendi a compilar códigos da linguagem C em várias etapas.
- ☐ Aprendi os passos para criar módulos com tipos abstratos de dados.
- ☐ Compreendi a diferença de tipos simples e ponteiros.
- ☐ Entendi o funcionamento da passagem de parâmetros por referência e por valor.
- ☐ Criei programas que usam tipos homogêneos por declaração e por ponteiros.
- ☐ Desenvolvi programas que usam tipos heterogêneos por declaração e por ponteiros.
- ☐ Criei um módulo de biblioteca com um Tipo abstrato de dados que usa tipos heterogêneos.

ESTUDO E PRÁTICA II

ESTRUTURAS BÁSICAS



Para trabalhar com programas, são necessários de meios de armazenamentos de dados que atendam determinadas especificações que surgem dos problemas. Neste bloco, você estudará mais sobre o assunto! Vamos lá!



E-book

As estruturas mais usadas são a pilha, a fila e a lista, assuntos que você acompanhará partir de agora. A partir dessas estruturas, você entenderá os fundamentos de controle sobre elementos em uma estrutura de dados. Em seguida, serão tratadas as mesmas estruturas, porém baseadas em listas encadeadas, aumentando um pouco a complexidade em relação a estas estruturas de dados. Aproveite e explore ao máximo esses assuntos!

As três estruturas básicas ou fundamentais, pilha, fila e lista, podem ser implementadas em vetores. No entanto, há uma outra forma de implementação dessas estruturas: usando listas encadeadas. O fato de mudar a estrutura básica sobre a qual essas estruturas serão implementadas oferecerá várias características que não estão presentes quando se usa vetores. E, a complexidade em sua manipulação também irá se alterar. Confira mais a seguir.



Estructuras estáticas básicas



ESTRUTURAS ESTÁTICAS BÁSICAS: CONCEITOS INICIAIS

Um dos elementos fundamentais em programas são as estruturas de dados. Um programa essencialmente manipula dados para gerar os resultados desejados. E, para isso, precisa contar com estruturas que possam permitir ou favorecer essa manipulação. Conheça agora três estruturas básicas ou fundamentais que são aplicadas em programação: a pilha, muito usada em sistemas operacionais e compiladores; a fila, também usada em sistemas operacionais e nos mais diversos problemas do dia a dia; e, finalmente, a lista, que serve de infraestrutura para diversas outras estruturas de dados.

Nas aplicações usadas no dia a dia, são empregadas algumas estruturas de dados para viabilizar seu tratamento e geração das informações pretendidas com a aplicação.

Confira, a seguir, as estruturas fundamentais mais usadas em aplicações e sua implementação de forma estática, que é sua forma de uso mais simples.





Precisamos organizar os dados para o uso pela aplicação. Em alguns casos, temos que respeitar a ordem como os dados aparecem; em outros, precisamos usar os dados mais recentes primeiro. Algumas estruturas podem trabalhar com os dados de tal forma que sua organização pode mudar, conforme o uso proposto. Isso faz com que a forma de agrupar os dados precise de alguns tipos de controle, que permita que essas necessidades sejam atendidas de forma plena.

Essencialmente, uma estrutura de dados deve conter um suporte, ou seja, um espaço de memória para armazenar os dados, e um conjunto de regras que define como esses dados podem ser manipulados. Backes (2016) define estrutura de dados assim: “[...] é uma forma de armazenar e organizar os dados de modo que possam ser usados de forma eficiente”. E, continua: “[...] uma estrutura de dados é um relacionamento lógico entre diferentes tipos de dados visando à resolução de determinado problema de forma eficiente”.

Percebe-se, nas definições, que a estrutura de dados tem como objetivo organizar os dados para que a aplicação possa manipulá-los de forma eficiente. Para determinar a eficiência de uma estrutura, esta pode ser avaliada pela complexidade que sua manipulação apresenta. Vamos começar discutindo brevemente o que é complexidade, para depois entendermos essas estruturas de dados básicas.

ANÁLISE ALGORÍTMICA

A análise algorítmica é uma forma de avaliar o desempenho de determinado algoritmo. Isso permite que possamos fazer escolhas para definir a melhor forma de resolver um problema. Isso é comumente feito pelas análises do melhor caso, pior caso ou caso médio. Acompanhe aqui a análise dos piores casos. Conforme Loudon (2000), a análise dos piores casos é a mais usada e também é conhecida como notação-O (diz-se big O). Loudon (2000) define complexidade computacional como “[...] a taxa de crescimento dos recursos (normalmente tempo) que um algoritmo necessita no que diz respeito ao tamanho dos dados que processa”. Com essa definição, o autor cita que “[...] a notação-O é uma expressão formal de complexidade de um algoritmo.”

Para avaliar a complexidade, que é feita em função da quantidade de dados envolvida, geralmente denominada n , Loudon (2000) apresenta algumas regras para avaliar essa taxa de crescimento. A primeira é que devemos desconsiderar fatores constantes. Por exemplo, se temos que ler um conjunto de dados de n elementos e, antes de fazê-lo, há 5 operações realizadas e mais 10 operações depois, somente consideramos n , ignoramos as 15 operações extras. A avaliação da complexidade se preocupa com o quanto cresce o custo da operação em função da quantidade de dados que aumenta. Como esse fator é constante, ele não é relevante nesta análise. Mesmo quando se tem um multiplicador, por exemplo, cada operação é realizada duas vezes sobre cada elemento, o que será considerado é n e não $2n$. Acompanhe alguns exemplos de código e sua complexidade.

```
for(i = 0; i < n; i++) {  
    (código ...)  
}
```

Neste caso, tem-se a complexidade n , na notação é $O(n)$.

```
for(i = 0; i < n; i++) {  
    for(j = 0; j < n; j++) {  
        (codigo ...)  
    }  
}
```

Neste caso, há uma complexidade n^2 , na notação é $O(n^2)$. Outro exemplo:

```
for(i = 0; i < n; i++) {  
    (código ...)  
    n = n/2;  
}
```

A cada passo no laço, a quantidade de operações é reduzida pela metade, aproximando-se do comportamento do logaritmo. Neste caso, tem-se complexidade $\log n$, na notação $O(\log n)$. As complexidades mais comuns apresentadas da menos complexa para a mais complexa são:



- $O(1)$ - Constante
- $O(\log n)$ - Logarítmica
- $O(n)$ - Linear
- $O(n \log n)$ - $n \log n$
- $O(n^2)$ - Quadrática
- $O(n^3)$ - Cúbica

Com isso, você já pode usar essa notação para avaliar a complexidade de algoritmos.

ESTRUTURAS ESTÁTICAS

Há três estruturas fundamentais, largamente usadas em computação: pilhas, filas e listas. Essas estruturas apresentam as restrições de acesso mais comuns que encontramos nos problemas computacionais. Conheça cada uma dessas estruturas construídas sobre uma estrutura estática, o vetor.

Pilha ou Stack

Uma das estruturas mais simples é a pilha ou Stack. Conforme Moraes (2001), a pilha “[...] é uma estrutura linear na qual todos os acessos (inserção, remoção ou consultas) são realizados em uma só extremidade, denominada TOPO”.

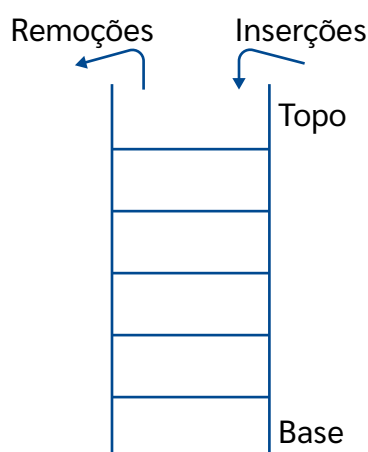
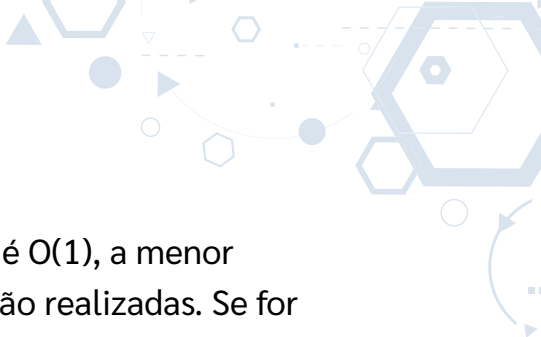


Figura 1 - Pilha
Fonte: Adaptado de Moraes (2001)

Devido às características observáveis da pilha, ela é chamada LIFO (*Last In First Out*), ou seja, o último a entrar é o primeiro a sair. Um exemplo para ilustrar uma pilha de forma mais adequada a seu conceito pode ser uma pilha de pratos. Quando se empilha pratos, é muito difícil acessar um prato no meio da pilha. Por isso, ao inserir novos pratos na pilha, eles são colocados no topo e, para removê-los, também o processo inicia pelo topo. A função que insere um elemento na pilha é chamada *push*, e a função que retira elementos da pilha é chamada *pop*. Por esta razão, algumas vezes chamamos a pilha de *push-pop*. Observe que o número de operações na inserção e na remoção são constantes.



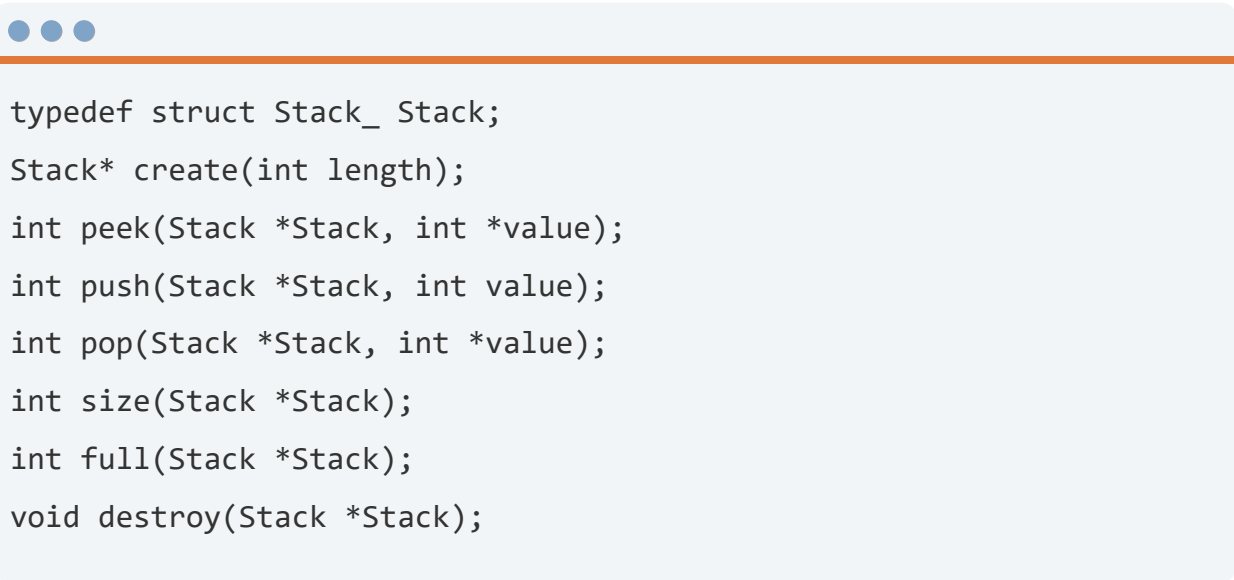
Com isso, dizemos que a complexidade dessas operações é $O(1)$, a menor complexidade possível. Não importa quantas operações são realizadas. Se for um número constante, dizemos que sua complexidade é $O(1)$. Também temos complexidade $O(1)$ nas funções *peek* e *size*, as quais você irá conhecer a seguir.

Essa é uma estrutura muito utilizada na computação. Um exemplo bem conhecido é o “Undo” (ctrl-z) ou “desfazer”, do Microsoft Word. As ações do usuário vão sendo empilhadas. Quando se solicita o desfaz, a última operação é desfeita, se solicitado novamente, a anterior, e assim por diante. Outra operação que usa pilhas é o *rollback* de banco de dados.

É possível observar que em operações em que a execução precisa ser realizada do último ao primeiro, a pilha é a estrutura mais adequada. Isso é muito comum em sistemas computacionais, o que torna a pilha muito usada.

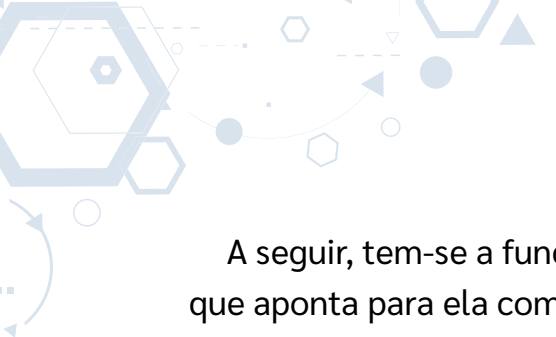
Criando um tipo abstrato de dados para a pilha, é preciso ter a implementação e o arquivo de cabeçalho dessa estrutura. É necessário lembrar que, sendo um arquivo compilado à parte, não podemos usá-lo com tipos simples. Então, a declaração do tipo terá que ser por ponteiro. Logo, a responsabilidade de criar a estrutura deve ser de uma função que foi compilada junto com o tipo abstrato de dados.

Confira o arquivo de cabeçalho de uma pilha de inteiros:



```
typedef struct Stack_ Stack;
Stack* create(int length);
int peek(Stack *Stack, int *value);
int push(Stack *Stack, int value);
int pop(Stack *Stack, int *value);
int size(Stack *Stack);
int full(Stack *Stack);
void destroy(Stack *Stack);
```

Você pode observar na primeira linha a definição parcial do tipo, para que o nome *Stack* possa ser usado pelo usuário da biblioteca na declaração de uma pilha.



A seguir, tem-se a função que cria uma pilha vazia, devolvendo um ponteiro que aponta para ela como retorno. Essa função recebe como parâmetro o tamanho da pilha a criar. Se a função apresentar erro, o ponteiro devolvido será NULL. Sua complexidade é $O(1)$.

A função *peek* consulta o elemento que está no topo da pilha, sem retirá-lo. Não há retorno do valor do topo, mas o ponteiro *value*, passado como parâmetro, recebe, dentro da função, esse valor do topo. Ele não pode ser o retorno no tipo simples *int* da função, porque, neste caso, o retorno será usado para identificar se houve erro na execução da função. Por exemplo, como se sabe que a pilha está vazia sem um retorno de erro? Se o retorno for diferente de zero, é porque houve erro na execução da função. Sua complexidade é $O(1)$.

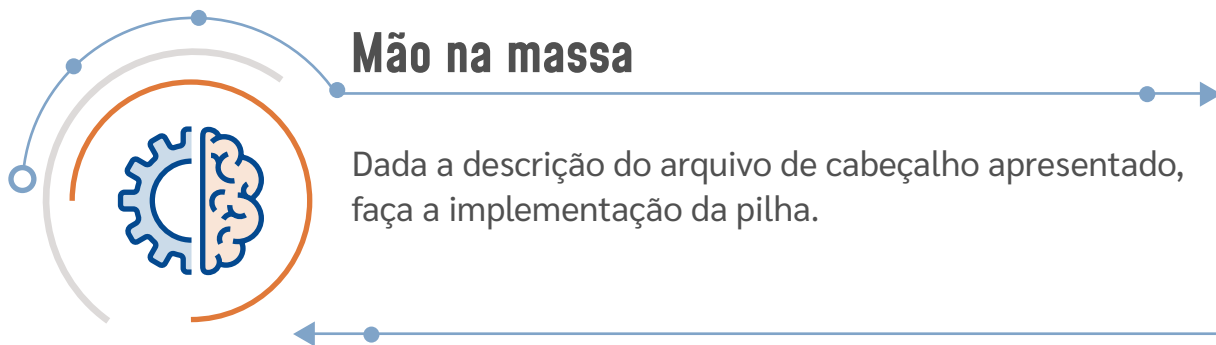
A função *push* insere o valor passado pelo parâmetro *value* na pilha. Se o retorno da função for zero, a inserção foi realizada com sucesso. Sua complexidade é $O(1)$.

A função *pop* remove o valor do topo da pilha e o armazena no ponteiro passado por parâmetro *value*. Se o retorno da função for zero, a remoção foi realizada com sucesso. Sua complexidade é $O(1)$.

A função *size* é usada para saber a quantidade de elementos na pilha. Não há como saber a quantidade de elementos em um vetor, a não ser que seja controlado durante a entrada e a saída de dados. Por isso, a necessidade dessa função. Sua complexidade é $O(1)$.

A função *full* é necessária para se saber se a pilha está cheia, pois como estamos usando um vetor como base para a pilha, há, por isso, um número de elementos máximo. É responsabilidade do tipo abstrato de dados controlar se a pilha está cheia com base em seu tamanho definido na criação. Sua complexidade é $O(1)$.

A função *destroy* é necessária, porque a lista é obtida pela função *create*, que faz a alocação de espaço para a lista. Este espaço não foi alocado pelo compilador e, por isso, precisa ser explicitamente liberado. Quem tem acesso à lista é o tipo abstrato de dados. Por isso, precisamos de sua função *destroy* para poder liberar o espaço alocado por *create*. Sua complexidade é $O(1)$.



Fila ou Queue

É difícil imaginar um contexto que não tenha algum tipo de fila sendo usado. Há filas para atendimento de pessoas, filas para execução de rotinas, filas de impressão, enfim, há filas para as mais diversas utilidades. A fila é muito útil pela sua característica de justiça. Quem chega primeiro é atendido primeiro. Moraes (2001) define fila como “[...] uma lista linear na qual todas as inserções são realizadas em uma das extremidades (fim da fila). E além disso, todas as remoções são realizadas em outra extremidade (início da fila)”. Dado essas características, o autor cita que essas estruturas são chamadas FIFO (*First In First Out*), ou seja, o primeiro que entrar será o primeiro a sair.



Figura 4 - Fila
Fonte: Do autor (2023)

Filas também são largamente utilizadas em computação. Quando implementadas em vetor, temos um problema que não tínhamos na pilha, que é a necessidade de deslocar seus elementos quando ocorre uma remoção.

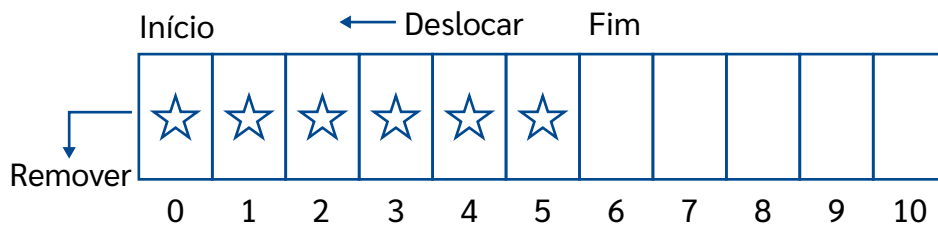


Figura 2 - Remoção da fila
Fonte: Do autor (2023)

Se esse deslocamento não for feito, ficarão vazios no início, o que impediria a entrada de novo elemento no fim quando todos os espaços do final forem preenchidos, mesmo havendo espaços vagos no início. Assim, temos a inserção com complexidade $O(1)$ e a remoção com complexidade $O(n)$. As funções *peek* e *size* também têm complexidade $O(1)$.

Na criação do tipo abstrato de dados para a fila, tem-se a implementação e o arquivo de cabeçalho. Confira, agora, o arquivo de cabeçalho de uma fila de inteiros:

```
typedef struct Queue_ Queue;
Queue* create(int length);
int peek(Queue *Queue, int *value);
int insert(Queue *Queue, int value);
int remove(Queue *Queue, int *value);
int size(Queue *Queue);
int full(Queue *Queue);
void destroy(Queue *Queue);
```

Observe, na primeira linha, a definição parcial do tipo, para que possa ser usado pelo usuário da biblioteca. A seguir, tem-se a função que cria uma fila vazia, devolvendo-a como retorno. Essa função recebe como parâmetro o tamanho da fila a criar. Se a função apresentar erro, o ponteiro devolvido será NULL. Sua complexidade é $O(1)$.

A função *peek* consulta o elemento que está no início da fila, sem retirá-lo. O retorno do valor do início da fila acontece pelo ponteiro *value* passado como parâmetro. Sua complexidade é $O(1)$.

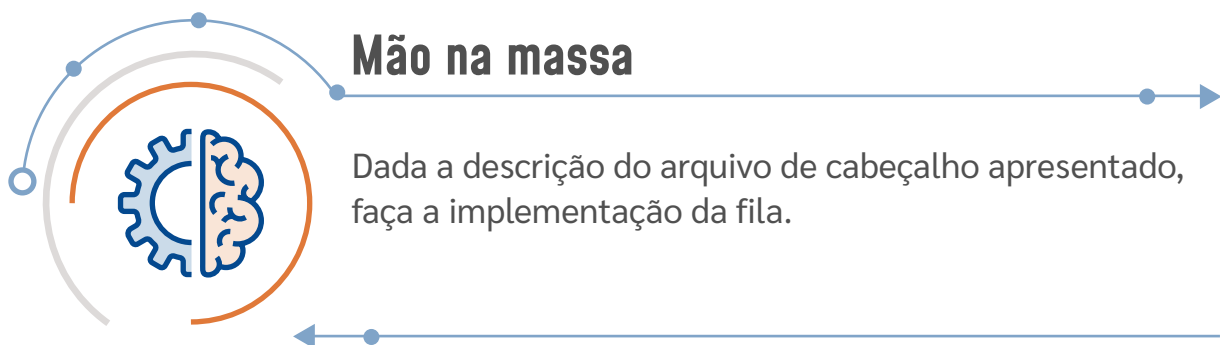
A função *insert* insere o valor passado pelo parâmetro *value* no final da fila. Se o retorno da função for zero, a inserção foi realizada com sucesso. Sua complexidade é $O(1)$.

A função *remove* retira o valor do início da fila e devolve no ponteiro passado por parâmetro *value*. Se o retorno da função for zero, a remoção foi realizada com sucesso. A novidade aqui fica por conta da sua complexidade, que neste caso é $O(n)$, pois como a função precisa deslocar todos os elementos da fila, a complexidade depende da quantidade de elementos.

A função *size* é usada para saber a quantidade de elementos na fila. Sua complexidade é $O(1)$.

A função *full* é usada para avaliar se a fila está cheia. Sua complexidade é $O(1)$.

A função *destroy* é usada para liberar a memória alocada pelo create. Sua complexidade é $O(1)$.



Lista

Certamente essa é a estrutura mais versátil dentre as que já foram apresentadas até aqui. Podemos ter inserção, consulta e remoção pelas extremidades ou em qualquer posição válida. Da mesma forma que na fila, não podemos deixar espaços livres. Então, a lista deve estar preenchida com seus espaços em uma das extremidades.

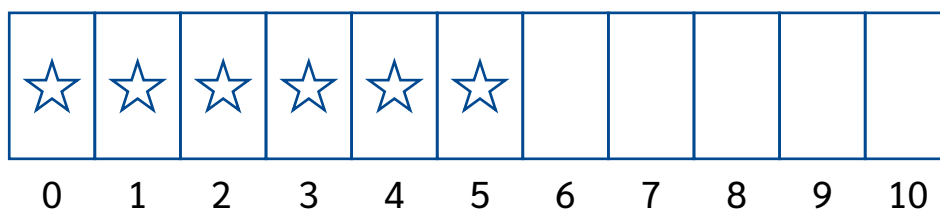


Figura 3 - Lista
Fonte: Do autor (2023)

Por exemplo, na figura anterior, as posições válidas para consultar ou remover são de 0 a 5, pois de 6 a 10 não possui elementos. No vetor, na verdade, haverá algo naquela posição, mas não é um valor que foi inserido por alguém na lista, ou seja, é um valor inválido. Para a inserção, as posições de 0 a 6 são válidas. Nas posições de 0 a 5, se houver inserção, deverá haver deslocamento dos elementos a partir daquela posição para a direita, para abrir espaço para o novo elemento a ser inserido. Já uma inserção na posição 6 não precisa de deslocamento, pois ela está vaga. A posição 7, no entanto, não é válida, pois, se inserirmos na posição 7, a 6 ficará com espaço, o que não é permitido.

Para entender melhor as posições válidas na lista, é preciso compreender como será controlado o conteúdo da lista. Quando se cria um vetor, é alocado espaço em memória para seus n possíveis elementos. Nessa alocação, o que havia naquele espaço de memória não é apagado, simplesmente reservado para o vetor. Por ser um vetor de inteiros, qualquer valor inteiro é válido de ser inserido em qualquer posição.

Mas, como podemos saber quantos elementos inserimos na lista?

Precisamos ter um controle de quantidade de elementos inseridos. Mas, se deixarmos espaços em branco no preenchimento da lista, como saberemos quais são os elementos que inserimos e quais são lixo, que já estavam na memória?

Poderíamos inicializar o vetor todo com zeros, mas zero também é um valor válido para ser inserido em um vetor de inteiros. Por isso, é preciso se preocupar com posições válidas de inserção e fazer os deslocamentos nas inserções e remoções.

Tratando de complexidade, na lista, a consulta terá complexidade $O(1)$, pois é uma consulta direta na posição desejada. A inserção no fim também terá complexidade $O(1)$, pois se trata de inserção em uma posição vaga. As demais funções de inserção terão complexidade $O(n)$, pois os elementos terão que ser deslocados. As funções de remoção terão complexidade $O(n)$, pois também precisarão deslocar os elementos da lista. Confira isso no código apresentado.

Na criação do tipo abstrato de dados para a lista, tem-se a implementação e o arquivo de cabeçalho. Acompanhe agora o arquivo de cabeçalho de uma lista de inteiros:

```
typedef struct List_ List;
List* create(int length);
int list_Head(List *list, int *value);
int list_Tail(List *list, int *value);
int list_position(List *list, int position, int *value);
int insert_Head(List *list, int value);
int insert_Tail(List *list, int value);
int insert_position(List *list, int position, int value);
int remove_Head(List *list, int *value);
int remove_Tail(List *list, int *value);
int remove_position(List *list, int position, int *value);
int size(List *list);
int full(List *list);
void destroy(List *list);
```

Iniciamos com a definição do tipo lista. Em seguida, temos a função que cria a lista devolvendo o ponteiro da lista criada.

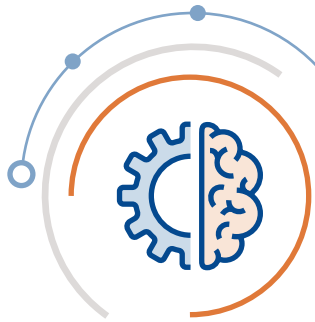
Como funções de consulta, temos *list_Head*, que consulta no início; *list_Tail*, que consulta no fim; e *list_position*, que pode consultar em qualquer posição válida. Nas consultas no início e no final, somente são necessários os parâmetros lista e o ponteiro para receber o valor lido. Na consulta em determinada posição, também é necessário o parâmetro da posição que se quer consultar.



Como funções de inserção, temos *insert_Head*, que insere no início; *insert_Tail*, que insere no fim; e *insert_position*, que pode inserir em qualquer posição válida. Nas inserções no início e no final, somente são necessários os parâmetros da lista e do valor a inserir. Na inserção em determinada posição, também é necessário o parâmetro da posição que se quer inserir.

Como funções de remoção, temos *remove_Head*, que retira do início; *remove_Tail*, que retira do fim; e *remove_position*, que pode retirar de qualquer posição válida. Nas remoções do início e do final, somente são necessários os parâmetros da lista e do ponteiro para receber o valor retirado. Na remoção em determinada posição, também é necessário o parâmetro da posição que se quer retirar.

A função *size* é usada para saber a quantidade de elementos existentes na lista. Sua complexidade é $O(1)$. A função *full* é usada para avaliar se a lista está cheia. Sua complexidade é $O(1)$. A função *destroy* libera a memória alocada por *create*. Sua complexidade é $O(1)$.

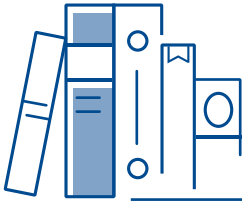


Mão na massa

Dada a descrição do arquivo de cabeçalho apresentado, faça a implementação da lista. Na galeria de vídeos, o vídeo: **Lista baseada em vetores** pode te ajudar.

As estruturas mais aplicadas computacionalmente são pilhas, filas e listas. Estas estruturas possuem regras de uso que as caracterizam e as tornam adequadas para o uso pretendido. A construção destas estruturas baseadas em vetor é de fácil implementação e a complexidade de suas funções de manipulação ficam entre $O(1)$ e $O(n)$, ou seja, complexidade constante ou linear.





REFERÊNCIAS

BACKES, A. R. **Estrutura de dados descomplicada:** em linguagem c. Rio de Janeiro: Elsevier, 2016.

MORAES, C. R. **Estrutura de dados e algoritmos:** uma abordagem didática. São Paulo: Berkeley Brasil, 2001.

LOUDON, K. **Dominando algoritmos com C.** Rio de Janeiro: Editora Ciência Moderna, 2000.



SENAI



E-book

Em diversos problemas computacionais, não temos como definir um tamanho máximo de elementos que serão manipulados. Uma possível solução é usar um vetor de tamanho muito maior que o necessário, como forma de garantir que não teremos problemas com limites de tamanho. Isso, no entanto, nem sempre é possível, pois a quantidade de elementos pode ser muito grande e tentar reservar um espaço ainda maior pode não ser praticável. Também temos o problema de consumir muita memória, mesmo quando esta não está em uso, pois o vetor consome todo o espaço alocado, independentemente de se ter elementos inseridos na estrutura. Confira mais a seguir.



Estruturas básicas usando listas encadeadas



LISTAS ENCADEADAS : CONCEITOS INICIAIS

Listas encadeadas, ou listas dinâmicas, são estruturas que podem crescer indefinidamente. Elas podem ser aplicadas como base para as estruturas básicas, como filas e pilhas, não necessitando delimitar um tamanho máximo para estas estruturas.

Loudon (2001) cita uma das mais importantes estruturas de dados, a lista encadeada. O autor aponta que servem a um propósito similar ao de *arrays*, agrupar dados, mas oferecem algumas vantagens sobre *arrays*. Ela é uma resposta aos problemas que temos com a implementação de listas com *arrays*, pois permite que a estrutura cresça na medida que for necessário e só ocupam o espaço de memória para elementos inseridos. A forma de trabalhar é diferente da que usamos com vetores, o que apresenta vantagens e desvantagens, conforme você poderá observar aqui.

LISTA ENCADEADA

Para alocar espaço para um vetor, o compilador precisa buscar um espaço de memória grande o suficiente para abrigar o tamanho todo do vetor.



```
Scopes = PageScall ;
foreach (Scopes as Scope) {
  $itemname = "curl/ha <loc>" . $url $scope=cat; %/loc/ha=cat/ha/
}

$categories = CategoryScall ;
foreach (Categories as Scat) {
  $itemname = "curl/ha <loc>" . route: name "categories.aha" ; [%tag] = $cat=ahag ; %/loc/ha=cat/ha/
}

// NO SCOPES
foreach (Scategories as Scat) {
  $itemname = "curl/ha <loc>" . route: name "scat/ha" ; [%tag] = $cat=ahag ; %/loc/ha=cat/ha/
}

// CATEGORY X SCOPES
$scatname = Catparam:with( relation: "category" )>get();
foreach (Scat as Scat) {
  if ($scat=category=public) {
    if ( $scat=category=show_sub_first on count/explode: domain "P" ; $scat=ahag ; %/loc/ha=cat/ha/
    $itemname = "curl/ha <loc>" . route: name "category.show" ; [%tag] = $cat=ahag ; %/loc/ha=cat/ha/
  }
}
```

Isso pode limitar o tamanho que pode ser alocado em determinado momento em uma aplicação. Essa é a razão para que estas estruturas sejam estáticas, pois não é possível garantir que os espaços após ao que foi alocado estarão livres se for necessário mais espaço.

Se não há necessidade que esses elementos fiquem em espaços contíguos, esse problema deixa de existir. Essa é a solução empregada com listas encadeadas. A estrutura que armazena o dado, ou elemento, deve conter um outro campo, que será um ponteiro. Este ponteiro irá apontar para o próximo elemento da lista, que pode estar em qualquer parte da memória.



Figura 1 - Lista encadeada
Fonte: Do autor (2023)

Podemos observar que a estrutura lista encadeada é composta de subestruturas, chamadas nodos, ou nós, que contêm um espaço para a informação que ela guarda e um ponteiro que aponta para o próximo elemento. Identifica-se que o último elemento foi alcançado, porque ele não possui próximo, representado na figura pelas três barras. A seguir, acompanhe duas maneiras de se criar nodos que podem compor uma lista encadeada: simples e duplos.

Nodo simples

O nodo simples é uma estrutura que tem suporte para guardar a informação a ser armazenada e um ponteiro que aponta para o próximo elemento.



Figura 2 - Nodo simples
Fonte: Do autor (2023)

Ao se criar um tipo abstrato de dados para o nodo, ocorrerá sua implementação e seu arquivo de cabeçalho. O arquivo de cabeçalho para um nodo que armazena um inteiro é apresentado a seguir:

```
typedef struct Node_ Node;
Node* create_node(int info);
void set_info(Node *node, int info);
int get_info(Node *node);
void set_next(Node *node, Node *next);
Node* get_next(Node *node);
void destroy(Node *node);
```

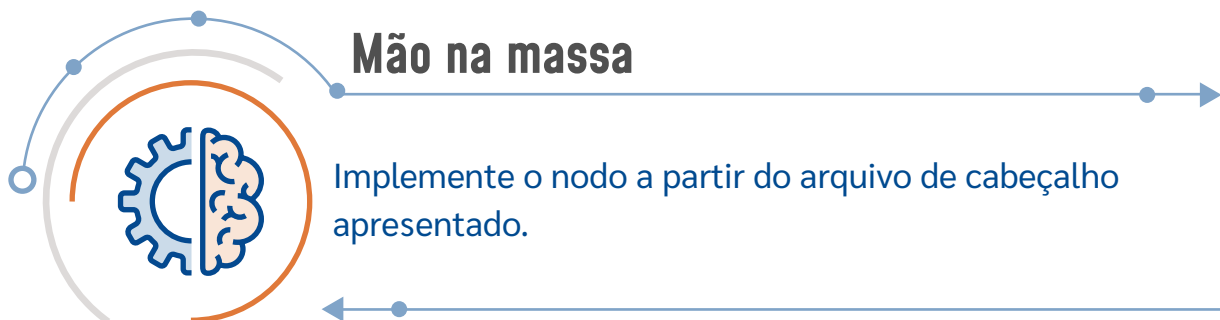
Analise agora esse arquivo de cabeçalho.

Na primeira linha, tem-se o *typedef* para a definição parcial do tipo *Node*. A seguir, consta o *create*, para a obtenção de um nodo novo. Sua complexidade é $O(1)$.

Para setar o dado armazenado no nodo, tem-se *set_info*, com complexidade $O(1)$. E, para recuperar o valor armazenado, consta a função *get_info*, também com complexidade $O(1)$.

Para setar o próximo nodo, tem-se a função *set_next*, com complexidade $O(1)$. E, para recuperá-lo, a função *get_next*, também com complexidade $O(1)$.

A função *destroy* libera a memória alocada pelo *create*, com complexidade $O(1)$.





Dica

Um nodo, quando criado, deve ter sua informação setada, e o próximo deve apontar para NULL. Dessa forma, não há como ocorrer erro na obtenção da informação. Este nodo pode ser incluído no final da lista, indicando que é o novo final da lista.

Nodo duplo

Em alguns casos, pode não ser suficiente conhecer quem é o próximo elemento. Poderá ser preciso saber também quem é o anterior. Neste caso, são necessários dois ponteiros: um para o próximo e um para o anterior.

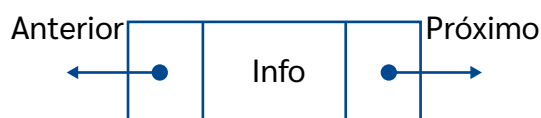
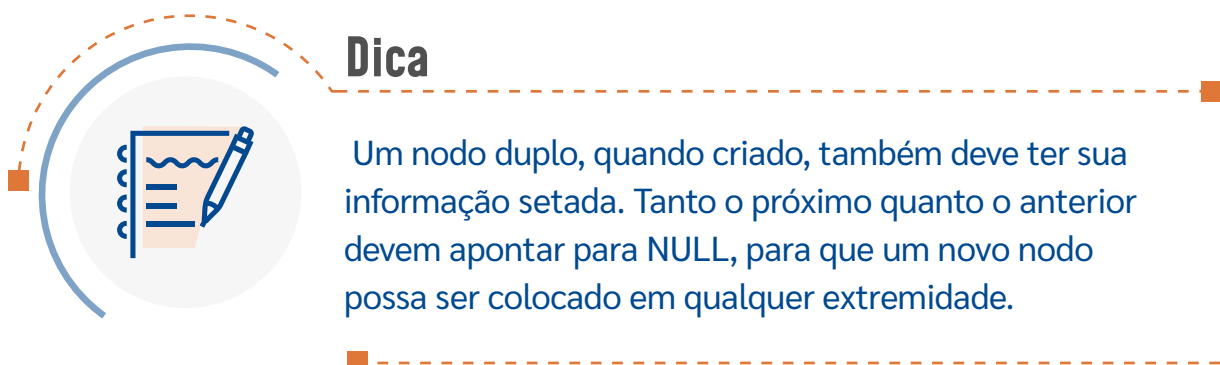
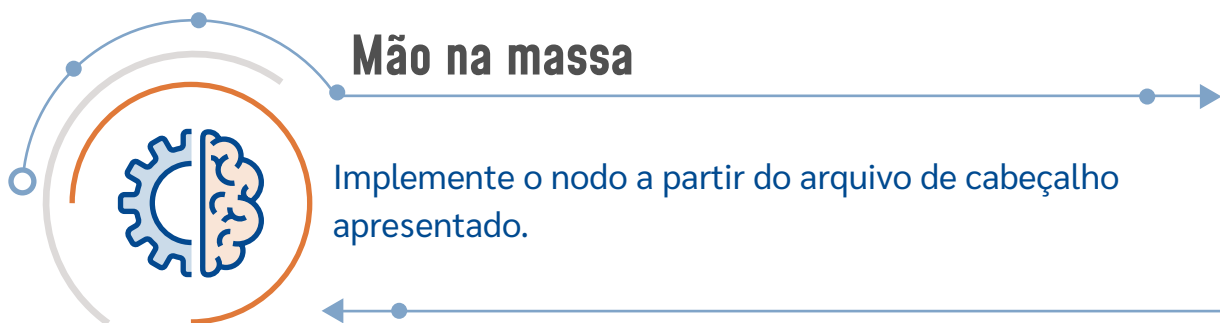


Figura 3 - Nodo duplo
Fonte: Do autor (2023)

Se criarmos um tipo abstrato de dados para o nodo duplo, teremos sua implementação e seu arquivo de cabeçalho. O arquivo de cabeçalho para um nodo duplo que armazena um inteiro é apresentado a seguir:

```
typedef struct DNode_ DNode;
DNode* create_dnode(int info);
void set_info(DNode *dnode, int info);
int get_info(DNode *dnode);
void set_next(DNode *dnode, DNode *next);
DNode* get_next(DNode *dnode);
void set_previous(DNode *dnode, DNode *previous);
DNode* get_previous(DNode *dnode);
void destroy(DNode *dnode);
```

Avaliando esse arquivo cabeçalho, podemos dizer que ele se assemelha muito ao de nodo simples. Apenas foi adicionado o ponteiro para o anterior e suas funções de setar e obter este ponteiro. A complexidade de suas funções será constante.



Lista encadeada de nodos simples

Uma lista encadeada de nodo simples permite crescimento indefinido. Sempre que se quiser inserir um novo elemento, é só criar o nodo e inseri-lo na posição desejada. Uma implementação com nodo simples é apresentada na galeria de vídeos, no vídeo: **Manipulando estruturas com nodos simples**. Confira.

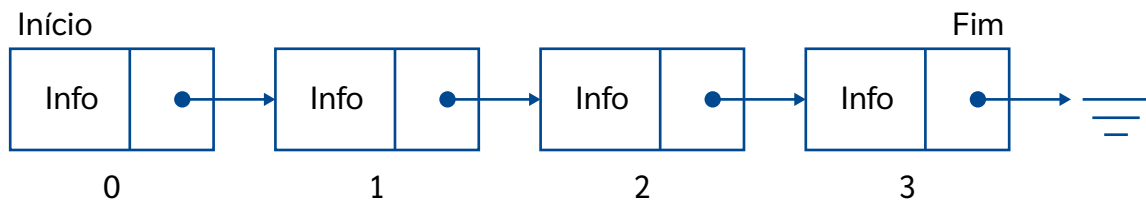


Figura 4 - Lista encadeada de nodo simples
Fonte: Do autor (2023)

Podemos observar que, para inserir no final, basta setar o ponteiro próximo do nodo da posição 3 para o novo nodo, ou seja, a complexidade é $O(1)$. Outras funções, no entanto, terão complexidade $O(n)$, pois será necessário navegar pelos elementos até chegar na posição de inserção desejada. A navegação nessa lista depende de um ponteiro para seu início e seguirá, de nodo em nodo, por meio dos ponteiros *próximo*. A definição da estrutura para essa lista seria:

```
struct List_ {  
    int size;  
    Node *Head;  
};
```

Em uma lista encadeada, podemos saber a quantidade de elementos dela por meio da navegação pelos nodos. Não há elementos inválidos.

Os elementos da lista são todos válidos, mas isso faria com que a função *size* tivesse complexidade $O(n)$. Para que isso não ocorra, devemos ter um campo na estrutura da lista que mantenha a informação de quantos elementos a lista possui, tornando a complexidade dessa função $O(1)$. O outro elemento na estrutura da lista é o nodo início, denominado *Head*, para que possamos ter acesso a lista. Na criação da lista, *size* deve ser zero e *Head* deve ser NULL.

Lista encadeada de nodos duplos

Para alguns problemas, poder navegar nos dois sentidos da lista implicará em redução da complexidade de algumas funções de manipulação da lista. Nesse caso, a estrutura de nodos duplos pode ser vantajosa.

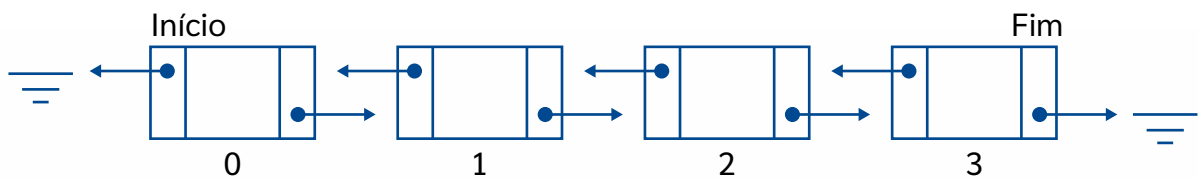
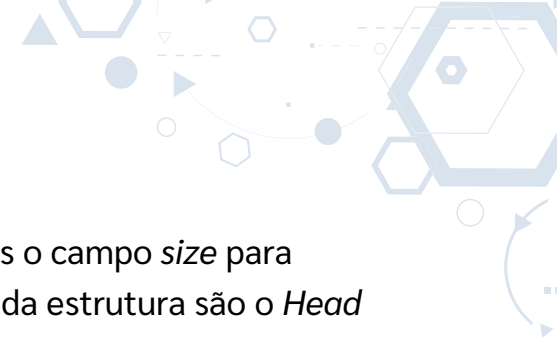


Figura 5 - Lista encadeada de nodo duplo
Fonte: Do autor (2023)

A primeira diferença que podemos observar nessa lista é que o ponteiro anterior do início aponta para NULL e o ponteiro próximo do fim também aponta para NULL, delimitando a lista. Podemos iniciar a navegação de qualquer uma das duas extremidades. Quando iniciamos do início, seguimos pelos ponteiros *próximo*. Quando iniciamos do fim, seguimos pelos ponteiros *anterior*. A definição da estrutura para essa lista seria:

```
struct DList_ {
    int size;
    DNode *Head;
    DNode *Tail;
};
```



Da mesma forma que na lista de nodo simples, usamos o campo *size* para reduzir a complexidade dessa função. Os outros campos da estrutura são o *Head* e o *Tail*, ponteiros para o início e o fim, respectivamente. Na criação da lista, *size* deve ser zero e *Head* e *Tail* devem ser NULL.

ESTRUTURAS FUNDAMENTAIS BASEADAS EM LISTAS ENCADEADAS

Para criar estruturas usando listas encadeadas, devemos considerar as funções que cada estrutura precisa e avaliar a complexidade das funções em cada tipo de lista encadeada.

Pilha

A pilha é uma estrutura que possui funções de baixa complexidade, $O(1)$, quando se usa vetores em sua implementação. A única razão para avaliarmos o uso de listas encadeadas como suporte para a pilha é pela possibilidade de redução de uso de memória quando essa pilha é necessária para um grande número de elementos. No caso do vetor, a pilha já inicia, mesmo vazia, consumindo todo o espaço de memória, como se estivesse cheia.

Criando um tipo abstrato de dados para a pilha que usa lista encadeada de nodo simples, precisamos ter a implementação e o arquivo de cabeçalho dessa estrutura. O arquivo de cabeçalho ficará quase igual ao da pilha implementada sobre o vetor, o que terá de diferença é a não existência da função *full*, pois essa pilha não precisa ter limite de tamanho. E, na função *create*, não é necessário parâmetro de tamanho da pilha. A função *destroy* terá sua complexidade aumentada, como você perceberá a seguir. Observe o arquivo de cabeçalho de uma pilha de inteiros:

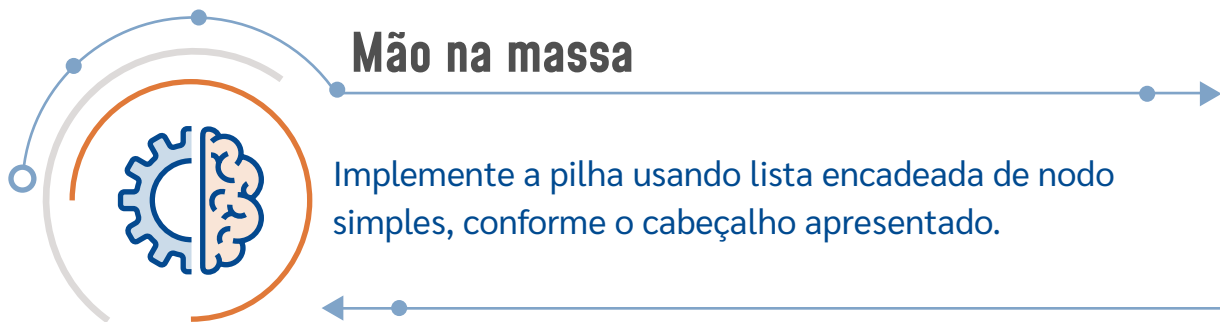
```
typedef struct Stack_ Stack;
Stack* create();
int peek(Stack *Stack, int *value);
int push(Stack *Stack, int value);
int pop(Stack *Stack, int *value);
int size(Stack *Stack);
void destroy(Stack *Stack);
```

Analisando as funções, é possível perceber que a função *create* simplesmente aloca espaço para a pilha, inicializando-a. Sua complexidade é $O(1)$.

- A função *peek* irá usar o campo *top* da estrutura da pilha, que aponta para o topo da pilha, tendo também complexidade $O(1)$.
- A função *push* irá setar o *próximo* do novo nodo para *top* e irá definir *top* para o novo nodo. Com isso, tem-se complexidade $O(1)$.
- A função *pop* irá guardar o *top* em um ponteiro, que será o elemento retirado, e setar o *top* para o seu próximo. Com isso, complexidade é $O(1)$.
- E, finalmente, a função *size* somente irá ler o campo *size* da pilha, tendo também complexidade $O(1)$.

Um detalhe deve ser observado na função *destroy*. A cada elemento inserido na pilha é criado um nodo novo, ou seja, é alocado memória para um nodo. No momento de destruir a pilha, antes de destruí-la, todos os nodos da pilha precisam ser destruídos antes, para não perdermos seus endereços. Do contrário, esse espaço ficaria alocado eternamente. Após destruir todos os nodos é que destruímos a pilha. Com isso, sua complexidade fica como $O(n)$.

Com essa análise, percebe-se que as complexidades das funções da pilha permanecem constantes, só alterada pela função *destroy*, que normalmente é pouco usada, não impactando na aplicação. O que realmente diferenciaria a implementação em vetor ou em lista encadeada seria o controle de uso de memória.



Fila

A fila baseada em vetor é uma estrutura que possui a função de inserção e consulta com complexidade $O(1)$, mas a de remoção como $O(n)$. Uma estratégia de implementação de fila com nodo simples pode ser observada na figura, a seguir.

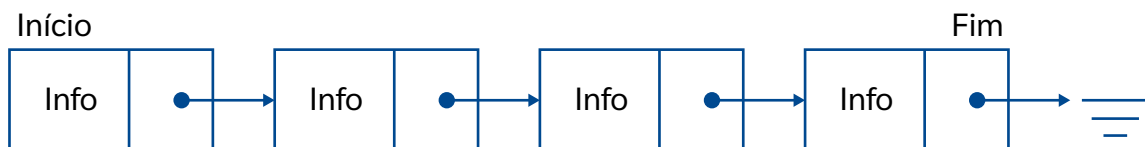


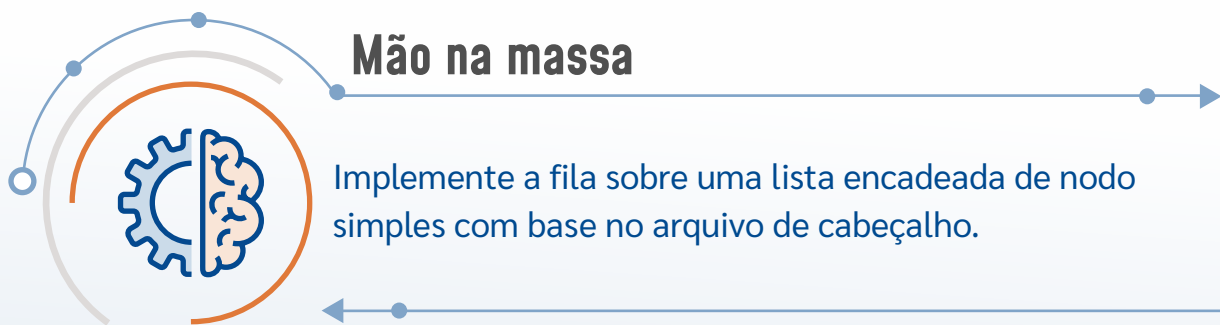
Figura 6 - Fila com nodo simples
Fonte: Do autor (2023)

A simples mudança de início e fim viabilizam as funções de inserção e remoção a operarem de forma mais simples. A inserção na fila ocorre no fim. Assim, para inserir, criamos um nodo novo e apontamos o *próximo* do Fim para o novo nodo, depois mudamos o Fim para apontar para o nodo novo. Com isso, sua complexidade fica $O(1)$. A remoção acontece pelo início. E, para isso, criamos um ponteiro temporário para apontar para o nodo Início, depois apontamos o Início para o seu próximo. Com isso, temos remoção com complexidade $O(1)$, mais simples do que usar um vetor. As demais funções, *peek* e *size*, também são de acesso direto, ficando com complexidade $O(1)$.

Percebe-se, com isso, que a complexidade da fila é menor com lista encadeada de nodo simples. A função *destroy* tem complexidade $O(n)$.

Criando um tipo abstrato de dados para a fila que usa lista encadeada de nodo simples, precisamos ter a implementação e o arquivo de cabeçalho dessa estrutura. O arquivo de cabeçalho ficará quase igual ao da fila implementada sobre o vetor, o que terá de diferença é a não existência da função *full*, pois essa fila não precisa ter limite de tamanho. E, na função *create*, não é necessário parâmetro de tamanho da fila. Observe o arquivo de cabeçalho de uma fila de inteiros:

```
typedef struct Queue_ Queue;
Queue* create();
int peek(Queue *Queue, int *value);
int insert(Queue *Queue, int value);
int remove(Queue *Queue, int *value);
int size(Queue *Queue);
void destroy(Queue *Queue);
```



Lista

A lista é uma estrutura que possui muitas funções, que precisam que os seus elementos conheçam seus vizinhos, tanto anteriores quanto posteriores. Por exemplo, se tivermos uma lista de nodo simples, para remover do início e do final, uma função ficará com complexidade $O(1)$ e a outra de $O(n)$, pois em uma obtemos o elemento diretamente, mas em outra isso não é possível, pois como não é possível voltar, não podemos redefinir o ponteiro do elemento anterior. Outras funções apresentam o mesmo problema. Por isso, para listas, a melhor opção deve ser a lista encadeada de nodo duplo.

As operações com nodo duplo precisam de mais atenção, pois há mais ponteiros que precisam ser setados corretamente. A chance de erro é muito maior do que a lista de nodos simples. O que justifica o uso da lista de nodo duplo é a redução de complexidade das funções.

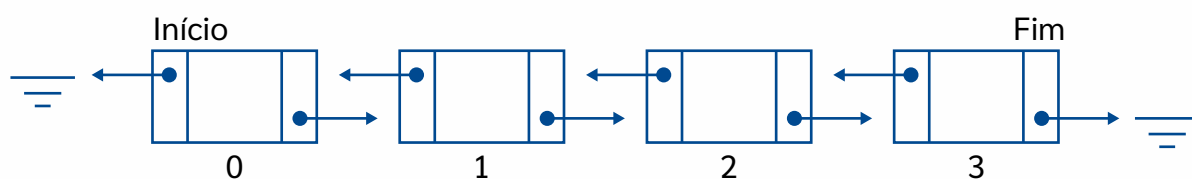


Figura 7 - Lista encadeada de nodo duplo
Fonte: Do autor (2023)

Ao criar um tipo abstrato de dados para a lista que usa encadeamento de nodo duplo, precisamos ter a implementação e o arquivo de cabeçalho dessa estrutura. O arquivo de cabeçalho ficará igual ao da lista implementada sobre o vetor, o que terá de diferença é a não existência da função *full*, pois essa lista não precisa ter limite de tamanho e na função *create* não é necessário parâmetro de tamanho da lista. Observe o arquivo de cabeçalho de uma lista de inteiros:

```

typedef struct List_ List;
List* create();
int list_Head(List *list, int *value);
int list_Tail(List *list, int *value);
int list_position(List *list, int position, int *value);
int insert_Head(List *list, int value);
int insert_Tail(List *list, int value);
int insert_position(List *list, int position, int value);
int remove_Head(List *list, int *value);
int remove_Tail(List *list, int *value);
int remove_position(List *list, int position, int *value);
int size(List *list);
void destroy(List *list);

```

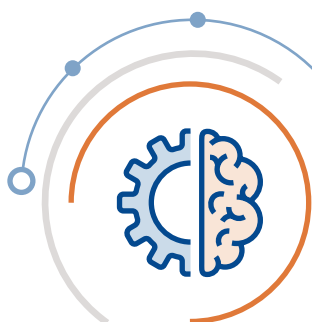
Tratando de complexidade, confira o quadro comparativo das complexidades das funções de uma lista baseada em vetor e baseada em lista encadeada.

FUNÇÃO	BASEADA EM VETOR	BASEADA EM LISTA ENCADEADA
<i>create</i>	O(1)	O(1)
<i>list_Head</i>	O(1)	O(1)
<i>list_Tail</i>	O(1)	O(1)
<i>list_position</i>	O(1)	O(n)
<i>insert_Head</i>	O(n)	O(1)
<i>insert_Tail</i>	O(1)	O(1)
<i>insert_position</i>	O(n)	O(n)
<i>remove_Head</i>	O(n)	O(1)

FUNÇÃO	BASEADA EM VETOR	BASEADA EM LISTA ENCADEADA
<i>remove_Tail</i>	O(1)	O(1)
<i>remove_position</i>	O(n)	O(n)
<i>size</i>	O(1)	O(1)
<i>destroy</i>	O(1)	O(n)

Quadro 1 - Lista baseada em vetor e encadeada
Fonte: Do autor (2023)

Dependendo das funções que mais são usadas na lista, conforme o problema que se quer resolver, uma será melhor do que a outra, com base nas complexidades apresentadas. Também devemos considerar o fato de a lista baseada em vetor ocupar o espaço máximo de memória, mesmo quando está vazia.

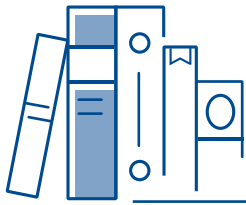


Mão na massa

Com base no arquivo de cabeçalho, implemente a lista usando a lista encadeada de nodo duplo. Na galeria de vídeos, o vídeo: **Manipulando estruturas de nodo duplo** pode te ajudar.

É possível implementar estruturas de mesmo comportamento, mas com infraestruturas diferentes. Cada tipo de infraestrutura oferece vantagens e desvantagens. Precisamos conhecer essas implementações, avaliar as vantagens e as desvantagens, conforme o tipo de problema que queremos resolver.

Uma avaliação criteriosa nos tipos de estruturas empregados na aplicação irá melhorar o seu desempenho e reduzir o volume de recursos que a aplicação irá exigir da máquina que a hospeda. Isso é um diferencial entre o desenvolvedor amador e o desenvolvedor profissional: a preocupação constante com a qualidade do que produz. Não basta funcionar, tem que funcionar bem. Siga com seus estudos e aprofunde seus conhecimentos!



REFERÊNCIAS

LOUDON, K. **Dominando algoritmos com C.** Rio de Janeiro: Editora Ciência Moderna, 2000.



SENAI



Videoflix



A seguir, confira alguns aspectos de código que são importantes para seus estudos. Acompanhe a apresentação da construção de uma lista baseada em *array* e de outra lista baseada em estrutura de nodos simples. Assista também a um vídeo demonstrando o uso de nodos duplos, que possuem uma manipulação mais detalhada. Explore e aproveite esse material.



Lista baseada em vetores



[https://player.vimeo.com/
video/800842192?h=7b69f2932a](https://player.vimeo.com/video/800842192?h=7b69f2932a)



Manipulando estruturas com nodos simples



[https://player.vimeo.com/
video/800842248?h=f4dd04fda3](https://player.vimeo.com/video/800842248?h=f4dd04fda3)



Manipulando estruturas de nodo duplo

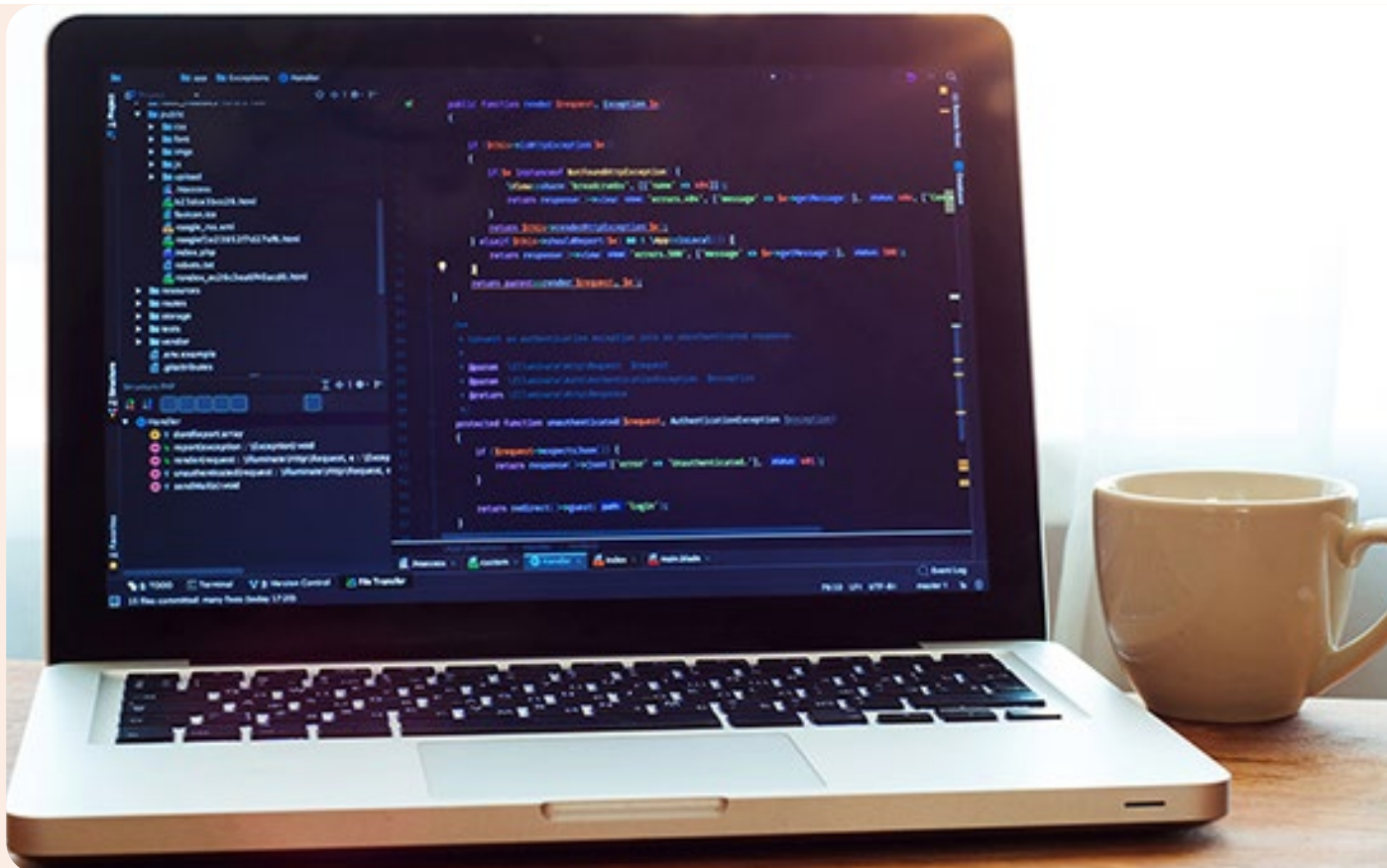


[https://player.vimeo.com/
video/800842232?h=805afefcce](https://player.vimeo.com/video/800842232?h=805afefcce)



Infocast

A seguir, confira alguns códigos comentados para deixar mais evidente o funcionamento das estruturas. Você estudará pilha sobre *array* e sobre listas dinâmicas. Assim, ficará mais fácil compreender a diferença na manipulação destas duas estruturas: estáticas e dinâmicas. Explore esse recurso e aprenda mais sobre esses temas.





Pilha sob vetor

Olá, estudante!

Confira a imagem que irá te auxiliar nesse podcast.



```
#include <stdlib.h>
#include "pilhaVetor.h"

struct Stack_ {
    int *elements;
    int length;
    int size;
};

Stack* create(int length) {
    Stack *stack = (Stack *) malloc(sizeof(Stack));
    stack->elements = (int *) malloc(sizeof(int) * length);
    stack->length = length;
    stack->size = 0;
    return stack;
}

int top(Stack *stack, int *value) {
    if(stack->size == 0) return 1;
    *value = stack->elements[stack->size - 1];
    return 0;
}

int push(Stack *stack, int value) {
    if(stack->size == stack->length) return 1;
    stack->elements[stack->size++] = value;
    return 0;
}
```



```
int pop(Stack *stack, int *value) {
    if(stack->size == 0) return 1;
    *value = stack->elements[(stack->size--)-1];
    return 0;
}
int size(Stack *stack) {
    return stack->size;
}
int full(Stack *stack) {
    if(stack->size == stack->length) return 0;
    return 1;
}
void destroy(Stack *stack) {
    free(stack->elements);
    free(stack);
}
```

```
#include <stdio.h>
int main() {
    int temp;
    int state;
    Stack *stack = create(5);
    for(int i = 10; i < 80; i+=10) {
        state = push(stack, i);
        if(state == 0) {
            top(stack, &temp);
            printf("Topo = %d\n", temp);
        } else {
            printf("Pilhas cheia\n");
        }
    }
}
```



```
for(int i = 10; i < 80; i+=10) {  
    state = pop(stack, &temp);  
    if(state == 0) {  
        printf("Removido = %d\n", temp);  
    } else {  
        printf("Pilhas vazia\n");  
    }  
}  
destroy(stack);  
}
```

Ao criar um tipo abstrato de dados para uma pilha, primeiro precisamos decidir se usaremos listas estáticas, como *arrays*, ou listas dinâmicas, que usam nodos. Vamos falar aqui sobre pilha sobre uma lista estática.

O primeiro passo é construir o arquivo de cabeçalho que precisará ser incluído no código de quem for usar a pilha. Este arquivo começa definindo um tipo *Stack* a partir de *struct Stack_*.

Depois, precisamos declarar a função que cria tipos pilha. Esta função precisa de um parâmetro que informe o tamanho da pilha, pois usaremos uma lista estática. Precisamos de uma função que confira quem está no topo da pilha, *top*. Seu único parâmetro é a pilha a ser lida. Para inserir e remover da pilha, temos as funções *push* e *pop*. A função *push* passa a pilha onde deve ser inserido o valor e o valor a ser inserido. A função *pop* passa a pilha de onde deve ser removido e um ponteiro para receber o valor removido. Temos ainda a função *size* e a função *full*, que servem para dizer a quantidade de elementos que existem na pilha e verificar se a pilha está cheia. Finalmente, temos a função para destruir a pilha, pois a alocação foi feita com o *create*. Isso encerra o arquivo de cabeçalho. A implementação vai implementar este cabeçalho.

A implementação precisa incluir a biblioteca *stdlib*, pois precisará usar a função *malloc* para criar a pilha. Também precisa incluir o cabeçalho que descreve a pilha, pois vai utilizar o tipo *Stack*.

A definição da pilha envolve a criação de três campos: o ponteiro *elements*, que irá guardar os dados como uma lista estática; um inteiro, para guardar o tamanho de criação da pilha, e um inteiro *size*, que irá guardar o número de valores que a lista contém.

Para criar a pilha, o primeiro passo é alocar um espaço para o ponteiro *Stack*. Como *elements* é um ponteiro, é necessário alocar espaço para que esse ponteiro seja uma lista estática, o tamanho da lista é o passado por parâmetro - *length*. Com os espaços alocados, iniciamos os demais campos: o *length*, com o valor do parâmetro; o *size*, com zero, indicando que a lista está vazia. Com a pilha criada, ela é retornada pela função.

A função *top* recebe a pilha e um ponteiro *value* como parâmetros. Como fará a leitura do primeiro elemento, primeiro precisa saber se a pilha não está vazia. Se não estiver vazia, atribui a *value* o valor na última posição preenchida do *array elements*. Com isso, a função retorna 0.

A função *pop* recebe a lista onde serão removidos o valor e o ponteiro receberá o valor removido. Verifica se a pilha está vazia; se não estiver, remove o valor na última posição ocupada do *array*, e então decrementa a quantidade de valores da pilha. Com isso, a função retorna 0.

A função *push* recebe a lista onde serão inseridos o valor e o valor a ser inserido. Verifica se a pilha está cheia; se não estiver, inclui o valor na primeira posição livre ao final do *array*, e então incrementa a quantidade de valores na pilha. Com isso, a função retorna 0.

As funções *size* e *full* são simples: *size* lê o campo *size* e *full*, compara *size* com *length*, para ver se a pilha está cheia.

A função *destroy* libera o espaço alocado pelo *create*. Primeiro libera o espaço do *array*, depois libera o espaço da pilha.

E com isso, temos a implementação da pilha. Espero que tenham gostado. Até mais!





Pilha sob lista encadeada

Olá, estudante!

Confira a imagem que irá te auxiliar nesse podcast.

```
Stack* create() {
    Stack *stack = (Stack *) malloc(sizeof(Stack));
    stack->top = NULL;
    stack->size = 0;
    return stack;
}

int top(Stack *stack, int *value) {
    if(stack->size == 0) return 1;
    *value = get_info(stack->top);
    return 0;
}

int push(Stack *stack, int value) {
    Node *node = create_node(value);
    set_next(node, stack->top);
    stack->top = node;
    stack->size++;
    return 0;
}

int pop(Stack *stack, int *value) {
    if(stack->size == 0) return 1;
    top(stack, value);
    Node *temp = stack->top;
    stack->top = get_next(temp);
    free(temp);
    stack->size--;
    return 0;
}
```



```
int size(Stack *stack) {
    return stack->size;
}

void destroy(Stack *stack) {
    Node *temp = stack->top;
    while(stack->top != NULL) {
        stack->top = get_next(temp);
        free(temp);
    }
    free(stack);
}

#include <stdio.h>
int main() {
    int temp;
    int state;
    Stack *stack = create(5);
    for(int i = 10; i < 80; i+=10) {
        state = push(stack, i);
        if(state == 0) {
            top(stack, &temp);
            printf("Topo = %d\n", temp);
        } else {
            printf("Pilhas cheia\n");
        }
    }
}
```



```
for(int i = 10; i < 100; i+=10) {  
    state = pop(stack, &temp);  
    if(state == 0) {  
        printf("Removido = %d\n", temp);  
    } else {  
        printf("Pilhas vazia\n");  
    }  
}  
destroy(stack);  
}
```

Ao criar um tipo abstrato de dados para uma pilha, primeiro precisamos decidir se usaremos listas estáticas, como *arrays*, ou listas dinâmicas, que usam nodos. Agora vamos falar aqui sobre pilhas sobre uma lista dinâmica.

O primeiro passo é construir o arquivo de cabeçalho que precisará ser incluído no código de quem for usar a pilha. Este arquivo começa definindo um tipo *Stack* a partir de struct *Stack_*.

Depois, precisamos declarar a função que cria tipos pilha. Esta função não precisa de parâmetro de tamanho para pilha, pois listas dinâmicas crescem indefinidamente, à medida que recebem novos nodos. Precisamos de uma função que confira quem está no topo da pilha, *top*. Seu único parâmetro é a pilha a ser lida. Para inserir e remover da pilha, temos as funções *push* e *pop*. A função *push* passa a pilha onde deve ser inserido o valor e o valor a ser inserido. A função *pop* passa a pilha onde o valor deve ser removido e um ponteiro para receber o valor removido. A função *size* serve para dizer a quantidade de elementos que existem na pilha. Observe que não precisamos de uma função para verificar se a pilha está cheia. Finalmente temos a função para destruir a pilha, pois a alocação foi feita com o *create*. Isso encerra o arquivo de cabeçalho. A implementação vai executar este cabeçalho.

Como usaremos uma lista dinâmica, precisaremos de um tipo abstrato de dados para o nodo. Teremos também um arquivo de cabeçalho para o nodo.

Este cabeçalho inicia com a definição do Node. Para o uso do nodo, precisamos de uma função de criação, *create_node*, que recebe um inteiro, para que o nodo já receba o valor que irá conter. Temos a função *get_info*, que recebe o parâmetro o nodo de onde iremos obter o valor. Temos a função *get_next*, que recebe o parâmetro do nodo do qual se quer obter o próximo nodo. E, finalmente temos o *set_next*, que recebe como parâmetros o nodo em que iremos definir o próximo e o nodo que será setado como próximo.

Na implementação do nodo, iniciamos incluindo o seu cabeçalho para poder usar o tipo Node e a biblioteca *stdlib*, para uso da função *malloc*.

A definição da estrutura do nodo contém dois campos: *value*, que é o valor que o nodo guarda e *next*, que é um ponteiro para apontar para o próximo nodo.

Para construir o nodo, temos um parâmetro, que é o valor que o nodo irá guardar. Iniciamos alocando o espaço para o nodo. Em seguida, são iniciados os campos *value*, com o parâmetro passado e *next* com NULL, para que não fique um endereço incorreto como lixo. Feito isso, o nodo é retornado pela função.

A função *get_info* recupera o valor armazenado no nodo passando por parâmetro. A função *get_next* recupera o nodo seguinte do nodo passado por parâmetro. A função *set_next* recebe o nodo que será manipulado e o nodo que será usado para definir o endereço do próximo.

Agora vamos falar sobre a implementação da pilha. Essa implementação precisa incluir a biblioteca *stdlib*, que precisará usar a função *malloc* para criar a pilha. Também precisa incluir o cabeçalho que descreve a pilha, pois irá utilizar o tipo *Stack*.

A definição da pilha possui um ponteiro para um nodo, que vai guardar o endereço do topo da pilha e um inteiro que irá manter a quantidade de valores da pilha. A criação da pilha envolve alocar o espaço para a estrutura da pilha, inicializar o ponteiro top com NULL e inicializar o *size* com 0. Com isso a função retorna um ponteiro para a pilha.

A função *top* recebe a pilha a ser consultada como parâmetro e um endereço de *value* para receber o valor lido. O primeiro passo é verificar se a pilha está vazia; não estando, o valor do nodo *top* é devolvido para *value* pela função *get_info*. A função retorna 0.

A função *push* recebe a pilha em que o valor será inserido por parâmetro, bem como o valor a ser inserido. Para inserir, criamos o nodo com o valor recebido por parâmetro e passamos a configurar os ponteiros da pilha.

Primeiro setamos o *next* do nodo novo com o endereço de *top*. Em seguida, definimos o *top* com o endereço do nodo novo. Depois incrementamos o *size* e a função retorna 0.

A função *pop* recebe por parâmetro a pilha da qual retiraremos o *value*, e um ponteiro para receber o valor retirado. Primeiro testamos se a pilha não está vazia, e não estando, atribuímos a *value* o valor do topo, usamos um ponteiro temporário para guardar o endereço do topo atual, setamos o topo com o endereço do próximo do ponteiro temporário e liberamos a memória do ponteiro temporário. Reduzimos o *size* e a função retorna 0.

A função *size* recebe a pilha por parâmetro e retorna a quantidade de valores na pilha.

A função que libera o espaço da lista recebe a lista por parâmetro. Antes de poder liberar o espaço da lista, temos que liberar o espaço de todos os seus nodos. Para isso, iniciamos um nodo temporário, que recebe o endereço do topo. Com um laço *while*, avançamos o topo para o próximo e liberamos o espaço do ponteiro temporário. Terminando o laço *while*, todos os nodos foram liberados. Agora podemos liberar a pilha.

Com isso temos a implementação da pilha.

Espero que tenham gostado.

Até a próxima!



Quero saber +

Gostou do assunto? Você pode aprender ainda mais sobre linguagens e módulos, buscando novos horizontes, sites, links, aplicativos e livros. Saiba mais lendo e conferindo os materiais a seguir.



Linguagem C

Clique no ícone, a seguir, e confira um pouco mais sobre a linguagem de programação C.



<https://dev.to/nfo94/resumo-basico-da-linguagem-c-para-logica-de-programacao-23o4>



Alguns exercícios em linguagem C

Clique no ícone, a seguir, e confira um github com vários exercícios resolvidos.



<https://www.huicode.com.br/p/exercicios-resolvidos-de-linguagem-c.html>



Resposta de exercícios com vários códigos em C

Clique no ícone, a seguir, e confira um github com vários exercícios resolvidos.



<https://github.com/misaelrezende/Exercicios-do-Livro-Linguagem-C-Completa-e-Descomplicada>





Resumindo

Agora chegou o momento de conferir um resumo dos principais conhecimentos aprendidos ao longo de seus estudos até aqui. Este resumo foi elaborado em formato de checklist, para que você assinale os itens que considera já ter desenvolvido e, caso sinta a necessidade, retome os estudos. Aproveite mais esta oportunidade de construção de saberes.

- ☐ Identifiquei a diferença entre linguagens de programação.
- ☐ Compreendi a diferença entre as formas de linguagem: interpretada, compilada ou híbrida.
- ☐ Pratiquei o uso de compiladores, produzindo seus artefatos intermediários.
- ☐ Elaborei programas como bibliotecas.
- ☐ Criei estruturas como pilhas, filas e pilhas.
- ☐ Criei bibliotecas de tipos abstratos de dados.
- ☐ Apliquei o uso de estruturas básicas em problemas.

PARA CONCLUIR



Parabéns! Você chegou ao final destes estudos. Clique no recurso a seguir e explore o infográfico que preparamos para você!

Ele traz os principais conceitos e definições que foram abordadas no decorrer de toda essa importante jornada de construção de conhecimentos. Esse é um breve resumo para você consultar sempre que sentir necessidade.



Estruturas Fundamentais

Confira agora um resumo dos principais conceitos abordados e retome sempre que sentir necessidade.

Análise de algoritmos:

Estudo do desempenho de algoritmos. Há várias formas de avaliar um algoritmo, tipicamente pelas análises de melhor caso, pior caso e caso médio. A mais usada e apresentada nesta unidade é a análise de pior caso.



Biblioteca:

Conjunto de tipos e funções relacionadas para um determinado fim. Vimos aqui bibliotecas de entrada e saída (*stdio*), de manipulação de memória (*stdlib*) e também construímos bibliotecas para TADs.



Estrutura dinâmica:

São aquelas em que o tamanho varia conforme são inseridos ou removidos os dados.



Estrutura estática:

São aquelas que possuem um tamanho definido, independentemente da quantidade de dados que estejam presentes em sua estrutura.



Fila:

Estrutura de dados onde a entrada é realizada em seu final e a saída em seu início, também chamada FIFO - *First In - First Out*.



Estruturas Fundamentais



Linguagem de programação:

É uma forma de expressar um processo ou sequência de passos, possuindo regras formais para isso.



Lista:

Estrutura de dados mais versátil. Permite entrada, saída e consulta em qualquer posição da estrutura.



Nodos:

São estruturas que permitem armazenar os dados e um ou dois endereços, que correspondem aos endereços de seus vizinhos.



Passagem por referência:

Diz-se passagem por referência quando um parâmetro de função é um endereço.

1001
0110
1010

Passagem por valor:

Diz-se passagem por valor quando o parâmetro de uma função é um valor.



Estruturas Fundamentais

Confira agora um resumo dos principais conceitos abordados e retome sempre que sentir necessidade.

Pilha:

Estrutura de dados em que suas operações de inserção e remoção ocorrem pela mesma extremidade, chamada topo.



Tipo abstrato de dados:

Corresponde a uma estrutura de dados capaz de armazenar algum tipo de informação e as funções que permitem a sua manipulação.



Tipos heterogêneos:

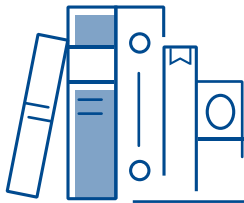
São estruturas em que os seus dados armazenados podem ser de tipos diferentes.



Tipos homogêneos:

São estruturas em que os seus dados armazenados são de mesmo tipo.





REFERÊNCIAS

AHO, A. V. et al. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamento da programação de computadores: algoritmos, pascal, c/c++ (padrão ansi) e java**. 3. ed. São Paulo: Pearson do Brasil Education, 2012.

BACKES, A. **Linguagem C: completa e descomplicada**. Rio de Janeiro: Elsevier, 2013.

BACKES, A. R. **Estrutura de dados descomplicada: em linguagem c**. Rio de Janeiro: Elsevier, 2016.

DEITEL, P.; DEITEL, H. **Como programar em C**. 6. ed. São Paulo: Pearson Prentice Hall, 2011.

FERREIRA, R. D. **Linguagem de programação**. Curitiba: Contentus, 2020.

KERNIGHAN, B. W.; RITCHIE, D. M. **C, a linguagem de programação: padrão ansi**. 2. ed. Rio de Janeiro: Campus, 1989.

LOUDON, K. **Dominando algoritmos com C**. Rio de Janeiro: Editora Ciência Moderna Ltda., 2000.

MORAES, C. R. **Estruturas de dados e algoritmos: uma abordagem didática**. São Paulo: Berkeley Brasil, 2001.

REESE, R. **Understanding and Using C Pointers**. Gravenstein Highway North: O'Reily Books, 2013.

SEBESTA, R. W. **Concepts of programming languages**. 10. ed. Upper Saddle River: Pearson, 2012.

SENAI